



# S<sup>4</sup>D: Style and Structure Similarity for Site Domains

S<sup>4</sup>D: Style and Structure Similarity for Site Domains: A system for website structure and visual style analysis, clustering, and comparison

Panagiotis Favvatas

Master's Thesis, Department of Informatics And Telecommunications  
University of Peloponnese, June 2025

Copyright © 2025 University of Peloponnese. All Rights Reserved

**S<sup>4</sup>D: Style and Structure Similarity for Site Domains:  
A  
system for website structure and visual style analysis,  
clustering, and comparison**

Panagiotis Favvatas

A thesis submitted in partial fulfillment  
of the requirements for the MSc in  
Computer Science

**Committee**

Professor, Spiros Skiadopoulos, Supervisor  
Professor, [Committee Member 1]  
Associate Professor, [Committee Member 2]

University of the Peloponnese  
2025

---

# Περίληψη

Αυτή η διπλωματική εργασία παρουσιάζει το **S<sup>4</sup>D** (Style and Structure Similarity for Site Domains), μια ερευνητική πλατφόρμα για την ανάλυση ομοιοτήτων μεταξύ ιστοσελίδων βάσει της δομής και του οπτικού τους στυλ. Το σύστημα επεξεργάζεται ιστοσελίδες αναλύοντας τη δομή του DOM και τις ιδιότητες CSS, συνδυάζοντάς τα σε μία ενιαία μορφή για συγκριτική αξιολόγηση.

Οι χρήστες μπορούν να εισάγουν οποιονδήποτε αριθμό URLs για ανάλυση. Κάθε σελίδα επεξεργάζεται ανεξάρτητα για την εξαγωγή δομικών και στυλιστικών χαρακτηριστικών. Κατά τη φάση υπολογισμού της ομοιότητας, οι συγκρίσεις πραγματοποιούνται σε επίπεδο domain, ανεξάρτητα από τον τρόπο ομαδοποίησης των URLs. Το σύστημα υποστηρίζει την ανάλυση σε διαφορετικά βάθη DOM (επίπεδα 1 έως 10).

Η ομαδοποίηση των στοιχείων πραγματοποιείται με τον αλγόριθμο HDBSCAN, ενώ η ποιότητα των ομάδων αξιολογείται με το DBCV. Τα απομονωμένα στοιχεία επανατοποθετούνται σε κατάλληλες ομάδες όταν είναι δυνατόν. Η πλατφόρμα επιτρέπει τη διεξαγωγή πολλαπλών πειραμάτων με διαφορετικά σύνολα URLs και παραμέτρους, καλύπτοντας ποι- κίλες ερευνητικές ανάγκες.

Το **S<sup>4</sup>D** υλοποιείται με React για το περιβάλλον χρήστη, Flask για τις διεπαφές API και Python για την επεξεργασία δεδομένων. Ενσωματώνει το Gnuplot για παραγωγή γραφημάτων και αρχείων κατάλληλων για ακαδημαϊκή χρήση. Ένα αρχείο αποτελεσμάτων επιτρέπει τη σύγκριση διαφορετικών πειραμάτων και την παρακολούθηση της προόδου.

Το σύστημα εξάγει HTML σελίδες με χρωματική κωδικοποίηση που αναδεικνύουν τις δομικές ομοιότητες, καθώς και πίνακες συσχετίσεων βασισμένους σε υπολογισμούς συνημιτόνου. Οι χρονικές μετρήσεις συμβάλλουν στην αξιολόγηση της απόδοσης του συστήματος. Οι δοκιμές δείχνουν ότι το **S<sup>4</sup>D** συγκρίνει και ομαδοποιεί με ακρίβεια στοιχεία ιστοσελίδων σε διαφορετικά domains, προσφέροντας ένα αξιόπιστο εργαλείο για δομική και οπτική ανάλυση.

Η πλήρης υλοποίηση του συστήματος είναι διαθέσιμη στο: [https://github.com/pfavvatas/lib\\_url\\_to\\_img](https://github.com/pfavvatas/lib_url_to_img)

---

# Abstract

This thesis presents S<sup>4</sup>D (Style and Structure Similarity for Site Domains), a research platform for analyzing similarities between websites based on their structure and visual style. The system examines web pages by processing their DOM structure and CSS styling, combining both into a unified representation for comparison.

Users can provide any number of URLs for analysis. Each page is processed independently to extract structural and styling information. During the similarity analysis phase, comparisons are performed at the domain level—regardless of how the URLs were grouped initially. The system supports analysis at multiple DOM depths, from level 1 to level 10.

After grouping similar elements using HDBSCAN and evaluating cluster quality with DBCV, the system reassigns outliers to the most appropriate clusters when possible. Users can perform multiple experiments using different URL sets and DOM levels to support varied research scenarios.

S<sup>4</sup>D is built with React for the interface, Flask for backend APIs, and Python for processing logic. It integrates Gnuplot to generate charts and data suitable for academic analysis. A results archive enables users to compare and track experiment outputs over time.

The system outputs color-coded HTML pages that highlight structural similarities, as well as similarity matrices based on cosine similarity. Timing metrics help evaluate system performance. Testing confirms that S<sup>4</sup>D reliably groups and compares website content across domains, offering a robust tool for structural and visual web analysis.

The full system implementation is available at: [https://github.com/pfavvatas/lib\\_url\\_to\\_img](https://github.com/pfavvatas/lib_url_to_img)

---



# Acknowledgements

I would like to express my sincere gratitude to all those who have contributed to the successful completion of this thesis.

First, I would like to thank Professor Spiros Skiadopoulos for his invaluable scientific guidance and mentorship throughout the course of this thesis. His expertise, constructive feedback, and immediate assistance whenever needed have been instrumental in shaping this research. His dedication to academic excellence has significantly contributed to my development as a researcher.

Furthermore, I am grateful to the academic staff and fellow researchers who have enriched my learning experience through valuable discussions and collaborative exchange of ideas.

I am deeply thankful to my family for their unconditional support, encouragement, and understanding throughout this academic journey. Their constant belief in my abilities has provided me with the strength to overcome challenges and pursue my goals.

Finally, I would like to acknowledge my friends and colleagues who have been a source of inspiration and motivation during these years. Their support and companionship have made this journey more meaningful and enjoyable.

---

# Contents

|                                                                   |            |
|-------------------------------------------------------------------|------------|
| <b>Περίληψη</b>                                                   | <b>i</b>   |
| <b>Abstract</b>                                                   | <b>iii</b> |
| <b>Acknowledgements</b>                                           | <b>v</b>   |
| <b>1 Introduction</b>                                             | <b>1</b>   |
| 1.1 Overview . . . . .                                            | 1          |
| 1.2 Problem Statement . . . . .                                   | 1          |
| 1.2.1 Web Structure Analysis Challenges . . . . .                 | 2          |
| 1.2.2 Research Motivation . . . . .                               | 3          |
| 1.3 Research Objectives . . . . .                                 | 3          |
| 1.4 Contributions . . . . .                                       | 4          |
| 1.5 Thesis Structure . . . . .                                    | 4          |
| <b>2 Theoretical Background</b>                                   | <b>5</b>   |
| 2.1 Web Parsing: DOM, CSS Analysis, and Layout Patterns . . . . . | 5          |
| 2.1.1 Structural Understanding of Web Documents . . . . .         | 5          |
| 2.1.2 Information Extraction from Web Pages . . . . .             | 6          |
| 2.1.3 DOM Tree Analysis and Processing . . . . .                  | 6          |
| 2.2 Data Modeling for Web . . . . .                               | 7          |
| 2.2.1 Web Data Representation Challenges . . . . .                | 7          |
| 2.2.2 JSON Structuring for Web Analysis . . . . .                 | 7          |
| 2.3 Similarity and Clustering Methods for Web Data . . . . .      | 7          |
| 2.3.1 Multi-dimensional Similarity Approaches . . . . .           | 7          |
| 2.3.2 Graph-based Similarity Measures . . . . .                   | 8          |
| 2.3.3 Role Vector Similarity . . . . .                            | 8          |
| 2.4 Clustering Algorithms for Web Data . . . . .                  | 8          |
| 2.4.1 Density-Based Clustering . . . . .                          | 8          |
| 2.4.2 Cluster Validation . . . . .                                | 9          |
| 2.4.3 Multi-level Clustering Analysis . . . . .                   | 9          |
| 2.5 Feature Engineering for Web Analysis . . . . .                | 9          |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| 2.5.1    | CSS Property Standardization . . . . .                     | 9         |
| 2.5.2    | Attribute Selection and Filtering . . . . .                | 9         |
| 2.5.3    | Vector Space Representation . . . . .                      | 10        |
| 2.6      | Similarity Measures for Web Data . . . . .                 | 10        |
| 2.6.1    | Cosine Similarity . . . . .                                | 10        |
| 2.6.2    | Earth Mover’s Distance (EMD) . . . . .                     | 10        |
| 2.6.3    | Hausdorff Distance . . . . .                               | 11        |
| 2.6.4    | Combined Similarity Metrics . . . . .                      | 11        |
| 2.7      | Applications and Use Cases . . . . .                       | 11        |
| 2.7.1    | Web Mining and Content Analysis . . . . .                  | 11        |
| 2.7.2    | User Interface Research . . . . .                          | 11        |
| 2.7.3    | Website Classification and Categorization . . . . .        | 11        |
| 2.7.4    | Design Pattern Analysis . . . . .                          | 11        |
| 2.8      | Research Gaps and Opportunities . . . . .                  | 12        |
| 2.8.1    | Integration of Multiple Analysis Dimensions . . . . .      | 12        |
| 2.8.2    | Scalability and Performance . . . . .                      | 12        |
| 2.8.3    | Dynamic Content Handling . . . . .                         | 12        |
| 2.8.4    | Cross-domain Generalization . . . . .                      | 12        |
| 2.9      | Literature Review Summary . . . . .                        | 12        |
| 2.10     | Transition to System Design . . . . .                      | 13        |
| <b>3</b> | <b>The S<sup>4</sup>D System</b>                           | <b>15</b> |
| 3.1      | System Overview . . . . .                                  | 15        |
| 3.2      | Technology Stack and Applied Tools . . . . .               | 16        |
| 3.2.1    | Backend Technologies . . . . .                             | 16        |
| 3.2.2    | Frontend Technologies . . . . .                            | 17        |
| 3.2.3    | System Integration . . . . .                               | 17        |
| 3.2.4    | Extended Technology Stack Details . . . . .                | 17        |
| 3.3      | Web Parsing (Levels + Clustering) . . . . .                | 18        |
| 3.3.1    | Automated Web Rendering . . . . .                          | 18        |
| 3.3.2    | Hierarchical DOM Tree Construction . . . . .               | 18        |
| 3.3.3    | Multi-Level DOM Analysis . . . . .                         | 19        |
| 3.3.4    | CSS Property Analysis and Normalization . . . . .          | 20        |
| 3.3.5    | Extended Web Parsing Capabilities . . . . .                | 23        |
| 3.4      | Data Modeling (Levels + Clustering) . . . . .              | 23        |
| 3.4.1    | Feature Engineering and Vector Representation . . . . .    | 23        |
| 3.4.2    | Pre-Clustering Data Modeling . . . . .                     | 24        |
| 3.4.3    | Vector Space Construction . . . . .                        | 25        |
| 3.4.4    | Vectorization and HDBSCAN Clustering Integration . . . . . | 25        |

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| 3.4.5    | Post-Clustering Data Modeling . . . . .                | 26        |
| 3.4.6    | Visualization of Clustered Elements . . . . .          | 26        |
| 3.5      | Data Mining . . . . .                                  | 27        |
| 3.5.1    | Transition Matrix Construction . . . . .               | 27        |
| 3.5.2    | Cluster Positional Statistics . . . . .                | 27        |
| 3.5.3    | Role Vector Construction . . . . .                     | 28        |
| 3.5.4    | Extended Data Mining Techniques . . . . .              | 28        |
| 3.6      | Similarity Analysis . . . . .                          | 29        |
| 3.6.1    | Multi-Dimensional Role Vector Construction . . . . .   | 29        |
| 3.6.2    | Complete Role Vector Specification . . . . .           | 30        |
| 3.6.3    | Best Setting Extraction . . . . .                      | 30        |
| 3.6.4    | Extended Similarity Analysis Framework . . . . .       | 31        |
| 3.6.5    | Best Level Selection Algorithm . . . . .               | 33        |
| 3.7      | Presentation of Resulting Analysis . . . . .           | 34        |
| 3.7.1    | Similarity Matrix Construction . . . . .               | 35        |
| 3.7.2    | Interactive Visualization Framework . . . . .          | 35        |
| 3.7.3    | Result Interpretation Guidelines . . . . .             | 36        |
| 3.7.4    | Extended Presentation and Export Features . . . . .    | 36        |
| 3.8      | Usage and Parameters . . . . .                         | 37        |
| 3.8.1    | System Configuration Parameters . . . . .              | 37        |
| 3.8.2    | Performance Tuning Guidelines . . . . .                | 38        |
| 3.8.3    | Usage Recommendations . . . . .                        | 38        |
| 3.9      | Performance Optimization and Scalability . . . . .     | 39        |
| 3.9.1    | Performance Monitoring . . . . .                       | 39        |
| 3.10     | Methodology Validation and Quality Assurance . . . . . | 39        |
| 3.10.1   | Algorithmic Robustness . . . . .                       | 39        |
| 3.11     | Methodology Summary . . . . .                          | 39        |
| <b>4</b> | <b>System Implementation</b>                           | <b>41</b> |
| 4.1      | System Architecture Overview . . . . .                 | 41        |
| 4.2      | Backend Implementation . . . . .                       | 41        |
| 4.2.1    | Core Processing Engine . . . . .                       | 41        |
| 4.2.2    | Multi-Level Analysis Implementation . . . . .          | 43        |
| 4.3      | API Implementation . . . . .                           | 43        |
| 4.3.1    | FastAPI Server Architecture . . . . .                  | 43        |
| 4.3.2    | File Serving Infrastructure . . . . .                  | 45        |
| 4.4      | Frontend Implementation . . . . .                      | 45        |
| 4.4.1    | React Application Architecture . . . . .               | 45        |
| 4.4.2    | User Interface Features . . . . .                      | 45        |

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| 4.4.3    | Integration with Backend Services . . . . .       | 46        |
| 4.5      | Performance Optimization Implementation . . . . . | 47        |
| 4.5.1    | Optimization Strategy . . . . .                   | 47        |
| 4.5.2    | Scalability Considerations . . . . .              | 47        |
| 4.6      | Deployment and Configuration . . . . .            | 47        |
| 4.6.1    | Development Environment Setup . . . . .           | 47        |
| 4.6.2    | Process Management and Execution . . . . .        | 48        |
| 4.6.3    | Production Deployment Considerations . . . . .    | 48        |
| 4.7      | Implementation Summary . . . . .                  | 48        |
| <b>5</b> | <b>Experiments and Evaluation</b>                 | <b>51</b> |
| 5.1      | Experimental Design . . . . .                     | 51        |
| 5.1.1    | Research Questions . . . . .                      | 51        |
| 5.1.2    | Evaluation Metrics . . . . .                      | 51        |
| 5.2      | Dataset Description . . . . .                     | 52        |
| 5.2.1    | Website Categories . . . . .                      | 52        |
| 5.3      | Experimental Results . . . . .                    | 53        |
| 5.3.1    | Clustering Quality Analysis . . . . .             | 53        |
| 5.3.2    | Performance Scalability . . . . .                 | 53        |
| 5.3.3    | System Performance Analysis . . . . .             | 56        |
| 5.4      | Comparative Analysis . . . . .                    | 56        |
| 5.4.1    | Algorithm Comparison . . . . .                    | 56        |
| 5.4.2    | Feature Importance Analysis . . . . .             | 56        |
| 5.5      | Statistical Validation . . . . .                  | 58        |
| 5.5.1    | Significance Testing . . . . .                    | 58        |
| 5.5.2    | Cross-Validation Results . . . . .                | 58        |
| 5.6      | Error Analysis and Limitations . . . . .          | 58        |
| 5.6.1    | Failure Case Analysis . . . . .                   | 58        |
| 5.6.2    | Quality Degradation Factors . . . . .             | 59        |
| 5.7      | Practical Implications . . . . .                  | 59        |
| 5.7.1    | Real-World Applicability . . . . .                | 59        |
| 5.7.2    | Performance Recommendations . . . . .             | 59        |
| 5.8      | Summary of Experimental Findings . . . . .        | 60        |
| <b>6</b> | <b>Results and Discussion</b>                     | <b>61</b> |
| 6.1      | Clustering Performance Results . . . . .          | 61        |
| 6.1.1    | Overall Clustering Quality . . . . .              | 61        |
| 6.1.2    | Domain-Specific Performance . . . . .             | 61        |
| 6.2      | Similarity Measurement Results . . . . .          | 63        |
| 6.2.1    | Individual Metric Performance . . . . .           | 63        |

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| 6.2.2    | Combined Metric Effectiveness . . . . .          | 64        |
| 6.3      | Computational Performance Results . . . . .      | 64        |
| 6.3.1    | Processing Time Analysis . . . . .               | 64        |
| 6.3.2    | Memory Usage Characteristics . . . . .           | 64        |
| 6.4      | Feature Engineering Analysis . . . . .           | 64        |
| 6.4.1    | Feature Component Contributions . . . . .        | 64        |
| 6.5      | Comparative Analysis Results . . . . .           | 66        |
| 6.5.1    | Performance Against Baselines . . . . .          | 66        |
| 6.6      | Case Study Results . . . . .                     | 66        |
| 6.6.1    | E-commerce Template Detection . . . . .          | 66        |
| 6.6.2    | News Website Pattern Analysis . . . . .          | 66        |
| 6.7      | System Validation Results . . . . .              | 66        |
| 6.7.1    | Expert Classification Correlation . . . . .      | 68        |
| 6.8      | Discussion of Findings . . . . .                 | 68        |
| 6.8.1    | Methodological Contributions . . . . .           | 68        |
| 6.8.2    | Practical Implications . . . . .                 | 68        |
| 6.8.3    | Technical Insights . . . . .                     | 69        |
| 6.9      | Limitations and Challenges . . . . .             | 69        |
| 6.9.1    | Dynamic Content Handling . . . . .               | 70        |
| 6.9.2    | Cross-Browser Variations . . . . .               | 70        |
| 6.9.3    | Cultural and Linguistic Factors . . . . .        | 70        |
| 6.9.4    | Scalability Considerations . . . . .             | 70        |
| 6.10     | Future Work Implications . . . . .               | 70        |
| 6.10.1   | Enhanced Dynamic Content Support . . . . .       | 70        |
| 6.10.2   | Cross-Browser Compatibility . . . . .            | 70        |
| 6.10.3   | Machine Learning Integration . . . . .           | 71        |
| 6.10.4   | Performance Optimization Enhancements . . . . .  | 71        |
| 6.11     | Results Summary . . . . .                        | 71        |
| <b>7</b> | <b>Conclusion and Future Work</b>                | <b>73</b> |
| 7.1      | Summary of Contributions . . . . .               | 73        |
| 7.1.1    | Primary Research Contributions . . . . .         | 73        |
| 7.2      | Key Research Findings . . . . .                  | 74        |
| 7.2.1    | Optimal Analysis Configuration . . . . .         | 74        |
| 7.2.2    | Performance and Accuracy Results . . . . .       | 74        |
| 7.2.3    | Domain-Specific Performance Patterns . . . . .   | 75        |
| 7.2.4    | Feature Importance Analysis . . . . .            | 75        |
| 7.3      | Theoretical and Practical Implications . . . . . | 75        |
| 7.3.1    | Theoretical Implications . . . . .               | 75        |

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| 7.3.2    | Practical Applications . . . . .                    | 76        |
| 7.4      | Limitations and Challenges . . . . .                | 77        |
| 7.4.1    | Technical Limitations . . . . .                     | 77        |
| 7.4.2    | Methodological Limitations . . . . .                | 77        |
| 7.5      | Future Work Directions . . . . .                    | 77        |
| 7.5.1    | Enhanced Dynamic Content Support . . . . .          | 78        |
| 7.5.2    | Cross-Browser and Cross-Platform Analysis . . . . . | 78        |
| 7.5.3    | Machine Learning Integration . . . . .              | 78        |
| 7.5.4    | Semantic and Content Integration . . . . .          | 79        |
| 7.5.5    | Scalability and Performance Enhancements . . . . .  | 79        |
| 7.6      | Broader Impact and Significance . . . . .           | 80        |
| 7.6.1    | Web Mining and Information Retrieval . . . . .      | 80        |
| 7.6.2    | Human-Computer Interaction . . . . .                | 80        |
| 7.6.3    | Digital Humanities and Social Sciences . . . . .    | 80        |
| 7.6.4    | Accessibility and Inclusive Design . . . . .        | 80        |
| 7.7      | Final Remarks . . . . .                             | 80        |
| <b>A</b> | <b>User Guide</b>                                   | <b>83</b> |
| A.1      | Getting Started . . . . .                           | 83        |
| A.1.1    | System Requirements . . . . .                       | 83        |
| A.2      | Interface Overview . . . . .                        | 83        |
| A.3      | Basic URL Analysis . . . . .                        | 83        |
| A.3.1    | Adding URLs . . . . .                               | 83        |
| A.3.2    | Selecting Analysis Levels . . . . .                 | 84        |
| A.3.3    | Running Analysis . . . . .                          | 84        |
| A.4      | Understanding Results . . . . .                     | 85        |
| A.4.1    | Site Similarity Analysis . . . . .                  | 85        |
| A.4.2    | Clustering Results . . . . .                        | 85        |
| A.4.3    | Generated HTML Files . . . . .                      | 85        |
| A.5      | Advanced Features . . . . .                         | 86        |
| A.5.1    | Experiment Configuration . . . . .                  | 86        |
| A.5.2    | Results Archive . . . . .                           | 86        |
| A.6      | Performance and Optimization . . . . .              | 86        |
| A.6.1    | System Efficiency . . . . .                         | 86        |
| A.6.2    | Performance Monitoring . . . . .                    | 87        |
| A.7      | Troubleshooting . . . . .                           | 87        |
| A.7.1    | Common Issues . . . . .                             | 87        |
| A.7.2    | Error Messages . . . . .                            | 88        |
| A.8      | Best Practices . . . . .                            | 88        |



|          |                                           |           |
|----------|-------------------------------------------|-----------|
| A.8.1    | Optimal Usage Patterns . . . . .          | 88        |
| A.8.2    | Result Interpretation . . . . .           | 88        |
| <b>B</b> | <b>API Documentation</b>                  | <b>89</b> |
| B.1      | API Overview . . . . .                    | 89        |
| B.1.1    | Base Configuration . . . . .              | 89        |
| B.2      | Core Processing Endpoints . . . . .       | 89        |
| B.2.1    | Process URLs . . . . .                    | 89        |
| B.2.2    | Process Clusters . . . . .                | 91        |
| B.3      | Experiment Management Endpoints . . . . . | 91        |
| B.3.1    | Run Experiment Batch . . . . .            | 91        |
| B.3.2    | List Experiment Results . . . . .         | 92        |
| B.3.3    | Get Experiment Details . . . . .          | 93        |
| B.4      | File Access Endpoints . . . . .           | 93        |
| B.4.1    | HTML File Serving . . . . .               | 93        |
| B.4.2    | Experiment File Downloads . . . . .       | 93        |
| B.4.3    | Generated Images . . . . .                | 94        |
| B.5      | Error Handling . . . . .                  | 94        |
| B.5.1    | Standard Error Response . . . . .         | 94        |
| B.5.2    | HTTP Status Codes . . . . .               | 94        |
| B.5.3    | Common Error Messages . . . . .           | 94        |
| B.6      | Rate Limiting and Performance . . . . .   | 94        |
| B.6.1    | Request Limits . . . . .                  | 94        |
| B.6.2    | Performance Considerations . . . . .      | 95        |
| B.7      | Authentication and Security . . . . .     | 95        |
| B.7.1    | Current Implementation . . . . .          | 95        |
| B.7.2    | Production Considerations . . . . .       | 95        |
| B.8      | Main API Endpoints . . . . .              | 95        |
| B.8.1    | URL Processing . . . . .                  | 96        |
| B.8.2    | Experiment Batch Processing . . . . .     | 96        |
| B.8.3    | Experiment Results Listing . . . . .      | 97        |
| B.8.4    | Experiment Details . . . . .              | 97        |
| B.9      | File Access . . . . .                     | 97        |
| B.10     | Error Handling . . . . .                  | 97        |
| <b>C</b> | <b>Technology Stack Details</b>           | <b>99</b> |
| C.1      | Technology Stack Overview . . . . .       | 99        |
| C.2      | Backend Technologies . . . . .            | 99        |
| C.2.1    | Core Programming Language . . . . .       | 99        |
| C.2.2    | Web Automation Framework . . . . .        | 99        |

|          |                                                 |            |
|----------|-------------------------------------------------|------------|
| C.2.3    | API Framework . . . . .                         | 100        |
| C.2.4    | Data Processing Libraries . . . . .             | 101        |
| C.2.5    | Machine Learning Libraries . . . . .            | 101        |
| C.2.6    | Image and Visualization Libraries . . . . .     | 102        |
| C.2.7    | Color Processing . . . . .                      | 102        |
| C.3      | Frontend Technologies . . . . .                 | 103        |
| C.3.1    | Core Framework . . . . .                        | 103        |
| C.3.2    | UI Component Library . . . . .                  | 103        |
| C.3.3    | Data Visualization . . . . .                    | 104        |
| C.3.4    | PDF Generation . . . . .                        | 104        |
| C.4      | Development and Deployment . . . . .            | 104        |
| C.4.1    | Package Management . . . . .                    | 104        |
| C.4.2    | Development Tools . . . . .                     | 104        |
| C.5      | Data Management . . . . .                       | 105        |
| C.5.1    | File System Storage . . . . .                   | 105        |
| C.5.2    | Data Processing Optimization . . . . .          | 105        |
| C.6      | Performance Optimization Technologies . . . . . | 106        |
| C.6.1    | Parallel Processing . . . . .                   | 106        |
| C.6.2    | Algorithm Optimization . . . . .                | 106        |
| C.7      | Browser Compatibility . . . . .                 | 106        |
| C.7.1    | Supported Browsers . . . . .                    | 106        |
| C.7.2    | Progressive Enhancement . . . . .               | 107        |
| C.8      | Security Considerations . . . . .               | 107        |
| C.8.1    | Input Validation . . . . .                      | 107        |
| C.8.2    | CORS Configuration . . . . .                    | 107        |
| C.9      | System Requirements . . . . .                   | 107        |
| C.9.1    | Minimum Requirements . . . . .                  | 107        |
| C.9.2    | Recommended Configuration . . . . .             | 108        |
| C.10     | Version Management . . . . .                    | 108        |
| C.10.1   | Dependency Versioning . . . . .                 | 108        |
| C.10.2   | Technology Lifecycle . . . . .                  | 108        |
| <b>D</b> | <b>Experimental Data</b>                        | <b>109</b> |
| D.1      | Experimental Design Overview . . . . .          | 109        |
| D.2      | Dataset Characteristics . . . . .               | 109        |
| D.2.1    | Test Website Categories . . . . .               | 109        |
| D.2.2    | URL Distribution . . . . .                      | 109        |
| D.3      | Performance Metrics . . . . .                   | 111        |
| D.3.1    | Processing Time Analysis . . . . .              | 111        |

|        |                                            |     |
|--------|--------------------------------------------|-----|
| D.3.2  | System Performance Metrics . . . . .       | 111 |
| D.4    | Clustering Quality Results . . . . .       | 111 |
| D.4.1  | DBCX Scores by Level . . . . .             | 111 |
| D.4.2  | Optimal Level Identification . . . . .     | 111 |
| D.5    | Similarity Analysis Results . . . . .      | 113 |
| D.5.1  | Cosine Similarity Distribution . . . . .   | 113 |
| D.5.2  | Multi-Metric Comparison . . . . .          | 113 |
| D.6    | Scalability Analysis . . . . .             | 113 |
| D.6.1  | URL Count vs Processing Time . . . . .     | 113 |
| D.6.2  | Memory Usage Patterns . . . . .            | 113 |
| D.7    | Experiment Validation Results . . . . .    | 115 |
| D.7.1  | Ground Truth Validation . . . . .          | 115 |
| D.7.2  | Cross-Validation Results . . . . .         | 115 |
| D.8    | Comparative Analysis . . . . .             | 115 |
| D.8.1  | Baseline Comparisons . . . . .             | 115 |
| D.8.2  | Feature Importance Analysis . . . . .      | 116 |
| D.9    | Error Analysis . . . . .                   | 116 |
| D.9.1  | Failure Case Analysis . . . . .            | 116 |
| D.9.2  | Quality Degradation Factors . . . . .      | 116 |
| D.10   | Performance Optimization Results . . . . . | 117 |
| D.10.1 | Optimization Techniques Impact . . . . .   | 117 |
| D.11   | Statistical Significance Tests . . . . .   | 117 |
| D.11.1 | Hypothesis Testing Results . . . . .       | 117 |
| D.12   | Reproducibility Information . . . . .      | 118 |
| D.12.1 | Experimental Configuration . . . . .       | 118 |
| D.12.2 | Data Availability . . . . .                | 118 |



# Chapter 1

## Introduction

### 1.1 Overview

The rapid evolution of the World Wide Web has resulted in an immense variety of website designs, layouts, and functionalities. With millions of active websites spanning diverse domains and purposes, understanding the structural and stylistic relationships between them has become increasingly valuable for applications such as web analytics, automated classification, design research, and interface standardization.

This thesis introduces S<sup>4</sup>D (Style and Structure Similarity for Site Domains), a comprehensive system for analyzing and comparing websites based on their Document Object Model (DOM) structure and Cascading Style Sheets (CSS) properties. S<sup>4</sup>D automates the process of extracting and encoding both structural and visual features of web pages and uses advanced clustering algorithms to group similar elements across different sites.

By transforming DOM and style elements into vector representations and applying density-based clustering techniques, the system reveals underlying structural patterns that may not be apparent through manual inspection. The result is a platform that supports large-scale comparisons between websites, highlighting domain-level similarities through color-coded visualizations and similarity matrices.

S<sup>4</sup>D addresses the growing need for scalable, automated tools that can help web developers, designers, and researchers understand trends in web structure and aesthetics. It provides an extensible foundation for future work in automated web categorization and interface analysis.

### 1.2 Problem Statement

The current landscape of web analysis tools primarily focuses on content-based analysis, often overlooking the significance of visual design patterns and structural similarities between websites. In the era of rapidly expanding web content, understanding the struc-

tural similarity between websites has become increasingly important for various applications including web mining, content analysis, and automated web processing. Traditional approaches face several critical challenges:

### 1.2.1 Web Structure Analysis Challenges

Modern web structure analysis presents several unique technical challenges that our S4D system specifically addresses:

- **DOM Complexity and Recursive Processing:** Modern websites contain deeply nested DOM structures with thousands of elements requiring recursive traversal algorithms. Our system handles complex parent-child relationships through unique ID generation and hierarchical tree processing, managing elements at multiple depth levels (1-10) while maintaining structural integrity during analysis.
- **CSS Style Diversity and Standardization:** The variety of CSS styling approaches across websites creates inconsistency in structural representation. Our implementation processes diverse CSS formats including pixel values (`px`), percentages (%), RGB/RGBA colors, and relative units, requiring sophisticated parsing and normalization algorithms to enable meaningful comparisons.
- **Attribute Extraction and Feature Engineering:** Extracting meaningful features from HTML elements involves processing hundreds of CSS properties per element. Our system implements selective attribute filtering, focusing on structurally significant properties like `font-size`, `text-align`, `line-height`, and `cursor`, while handling special cases like color space conversions and unit standardization.
- **Scalability and Performance Optimization:** Processing large numbers of websites requires efficient algorithms and memory management. Our system implements domain-based processing modes, ChromeDriver optimization, and parallel data processing to handle batch website analysis while minimizing computational overhead and ensuring reliable performance across different scales.
- **Multi-dimensional Similarity Computation:** Defining meaningful similarity measures for web structures involves multiple algorithmic challenges. Our approach combines HDBSCAN clustering with DBCV (Density-Based Cluster Validation) scoring [6], cosine similarity matrices, and role vector analysis to capture structural, visual, and positional relationships between website elements.
- **Clustering Parameter Optimization:** Achieving optimal clustering results requires dynamic parameter tuning. Our system implements grid search optimization over cluster size ranges and selection epsilon values, with automatic outlier reassignment to nearest cluster centroids based on Euclidean distance calculations.

- **Cross-Domain Pattern Recognition:** Identifying design trends across different domains requires sophisticated comparison frameworks. Our implementation uses domain-level aggregation, transition matrices between design elements, and combined similarity metrics (cosine, EMD, Hausdorff) to enable meaningful cross-domain analysis.

### 1.2.2 Research Motivation

The need for automated web structural analysis stems from several key factors:

- **Content Management:** Organizations need tools to understand and manage large collections of web content based on structural and visual characteristics.
- **Design Pattern Analysis:** Web developers and designers benefit from understanding common structural patterns and visual trends across websites in their domain.
- **Accessibility Evaluation:** Automated structural analysis can help identify accessibility issues and design inconsistencies across multiple web properties.
- **Web Analytics Enhancement:** Structural similarity analysis provides deeper insights into website categorization and user experience patterns beyond traditional content-based metrics.

## 1.3 Research Objectives

The primary objectives of this research are:

1. To develop a robust system for extracting and analyzing structural and visual features from websites
2. To create effective similarity metrics that can accurately measure relationships between different websites
3. To implement clustering algorithms that can group websites based on their design and structural characteristics
4. To provide a comprehensive comparison framework for website analysis across different domains
5. To validate the effectiveness of the proposed system through extensive testing and evaluation

## 1.4 Contributions

This thesis makes the following key contributions:

- **Novel feature extraction methodology:** Development of comprehensive techniques for extracting both structural and visual features from websites
- **Advanced similarity metrics:** Introduction of new metrics specifically designed for website comparison
- **Integrated clustering approach:** Implementation of clustering algorithms optimized for website analysis
- **Comprehensive evaluation framework:** Establishment of evaluation metrics and datasets for assessing website similarity systems
- **Practical applications:** Demonstration of real-world applications in web design analysis and automated categorization

## 1.5 Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter 2: Literature Review** presents a comprehensive review of existing approaches to website analysis, similarity measurement, and clustering techniques relevant to web data.
- **Chapter 3: Methodology** describes the proposed S<sup>4</sup>D system architecture, including feature extraction methods, similarity metrics, and clustering algorithms.
- **Chapter 4: Implementation** details the technical implementation of the system, including data structures, algorithms, and software architecture decisions.
- **Chapter 5: Experimental Design** outlines the experimental methodology, datasets used, and evaluation metrics employed to assess the system's performance.
- **Chapter 6: Results and Analysis** presents the experimental results, performance analysis, and comparison with existing approaches.
- **Chapter 7: Discussion** provides an in-depth discussion of the findings, limitations, and implications of the research.
- **Chapter 8: Conclusion and Future Work** summarizes the key findings, contributions, and suggests directions for future research.



# Chapter 2

## Theoretical Background

This chapter provides a comprehensive review of the theoretical foundations that underpin the S<sup>4</sup>D system, covering web parsing techniques, data modeling approaches, and similarity and clustering methods specifically applicable to web data analysis. The review examines foundational work in web structure analysis, similarity measurement, clustering algorithms, and related applications that inform the development of the S<sup>4</sup>D system.

### 2.1 Web Parsing: DOM, CSS Analysis, and Layout Patterns

Web parsing encompasses the comprehensive extraction and analysis of web page structural elements, including Document Object Model (DOM) traversal, Cascading Style Sheets (CSS) property analysis, element placement patterns, and XPath-based content extraction. Modern web parsing has evolved significantly since the early days of the World Wide Web, requiring sophisticated approaches to handle complex DOM structures and diverse CSS styling approaches.

#### 2.1.1 Structural Understanding of Web Documents

Chen et al. [1] introduced WebSRC, a comprehensive dataset for web-based structural reading comprehension that highlights the importance of understanding both semantic content and structural organization of web pages. Their work demonstrates that effective web page understanding requires systems to comprehend not only textual semantics but also the hierarchical structure of HTML documents. The WebSRC dataset, comprising 400K question-answer pairs from 6.4K web pages with corresponding HTML source code and screenshots, establishes a benchmark for structural web understanding tasks.

Building on the importance of structural representation, Deng et al. [2] proposed DOM-LM, a novel approach for learning generalizable representations of HTML docu-

ments. Their work addresses limitations of existing approaches by encoding both text and DOM tree structure using transformer-based encoders. DOM-LM demonstrates that combining textual content with structural information leads to superior performance in webpage understanding tasks, including attribute extraction, open information extraction, and question answering. The model shows particularly strong generalization capabilities in few-shot and zero-shot settings, making it suitable for real-world applications with limited labeled data.

### 2.1.2 Information Extraction from Web Pages

Lockard et al. [4] tackled the challenge of zero-shot relation extraction from semi-structured webpages through their ZeroShotCeres system. Their approach recognizes that textual semantics in web documents are often augmented with visual elements including layout, font size, and color. The system employs graph neural networks to build rich representations of text fields and their relationships, enabling generalization to previously unseen website templates. This work is particularly relevant to the S<sup>4</sup>D system’s goal of analyzing structural and visual similarities across different website domains.

The challenge of web-scale information extraction was addressed by Gulhane et al. [3] through their Vertex system. Vertex represents one of the first systems to achieve high-precision information extraction at web scale, processing over 250 million records from more than 200 websites. The system’s approach to grouping similar structured pages, learning robust XPath-based extraction rules, and detecting site changes provides valuable insights for large-scale web analysis systems like S<sup>4</sup>D.

### 2.1.3 DOM Tree Analysis and Processing

The Document Object Model (DOM) serves as the foundation for web page structure analysis. Recent advances in DOM-based analysis focus on extracting meaningful patterns from the hierarchical organization of HTML elements. The S<sup>4</sup>D system builds upon these foundations by implementing recursive DOM traversal algorithms that capture both structural and stylistic properties at multiple depth levels.

Current approaches to DOM analysis often struggle with the complexity and variability of modern web pages. The S<sup>4</sup>D system addresses these challenges through its multi-level analysis framework, which processes DOM structures at depths ranging from 1 to 10 levels, allowing for fine-grained structural comparison while maintaining computational efficiency.

## 2.2 Data Modeling for Web

Data modeling for web applications involves the systematic representation of web page elements, their attributes, and relationships in structured formats suitable for computational analysis. This includes the transformation of HTML/CSS data into machine-readable formats, the creation of inverted indices for efficient property lookup, and the development of hierarchical data structures that preserve DOM relationships while enabling scalable analysis.

### 2.2.1 Web Data Representation Challenges

Modern web data modeling faces several key challenges including the heterogeneous nature of HTML structures, the diversity of CSS styling approaches, and the need for normalized representations that enable meaningful comparisons across different websites and design frameworks.

### 2.2.2 JSON Structuring for Web Analysis

The structured representation of web data through JSON formats provides a foundation for systematic analysis. This includes the creation of hierarchical document models, attribute-value mappings, and metadata integration that supports both individual element analysis and cross-document comparisons.

## 2.3 Similarity and Clustering Methods for Web Data

Similarity measurement and clustering for web data represent specialized applications of machine learning techniques adapted for the unique characteristics of web content. The challenge lies in developing metrics that can capture the multifaceted nature of web page similarity, including structural, visual, and semantic dimensions while handling the high-dimensional and sparse nature of web feature spaces.

### 2.3.1 Multi-dimensional Similarity Approaches

Traditional similarity measures focus primarily on textual content, often overlooking the rich structural and visual information present in web pages. The S<sup>4</sup>D system advances this field by implementing a comprehensive similarity framework that considers multiple dimensions:

- **Structural Similarity:** Based on DOM tree organization and element hierarchies
- **Visual Similarity:** Derived from CSS properties and styling attributes

- **Positional Similarity:** Considering element placement and layout patterns
- **Semantic Similarity:** Incorporating content-based analysis where applicable

### 2.3.2 Graph-based Similarity Measures

Graph neural network approaches, as demonstrated in ZeroShotCeres [4], provide powerful frameworks for capturing complex relationships between web page elements. These approaches represent web pages as graphs where nodes correspond to DOM elements and edges represent structural or semantic relationships.

### 2.3.3 Role Vector Similarity

The S<sup>4</sup>D system introduces role vector similarity, which characterizes elements based on their functional roles within the document structure. This approach extends beyond simple structural matching by considering how elements relate to their neighbors and contribute to the overall page organization.

## 2.4 Clustering Algorithms for Web Data

Web data presents unique challenges for clustering algorithms due to its high dimensionality, sparsity, and heterogeneous nature. Effective clustering of web pages requires algorithms that can handle these characteristics while producing meaningful and interpretable results.

### 2.4.1 Density-Based Clustering

The S<sup>4</sup>D system employs HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise) [5] as its primary clustering algorithm. HDBSCAN provides several advantages for web data clustering:

- **Automatic cluster number detection:** Eliminates the need for pre-specifying the number of clusters
- **Noise handling:** Effectively identifies and separates outlier web pages
- **Hierarchical structure:** Reveals nested clustering patterns common in web data
- **Parameter robustness:** Less sensitive to parameter selection compared to traditional clustering methods

### 2.4.2 Cluster Validation

The system implements DBCV (Density-Based Cluster Validation) scoring [6] to assess clustering quality. DBCV provides a measure of cluster validity that considers both within-cluster cohesion and between-cluster separation, making it particularly suitable for evaluating the quality of web page groupings.

### 2.4.3 Multi-level Clustering Analysis

A key innovation in the S<sup>4</sup>D system is its multi-level clustering approach, which performs clustering analysis at different DOM depth levels. This approach recognizes that different levels of structural granularity may reveal different clustering patterns, allowing for comprehensive analysis of website similarities.

## 2.5 Feature Engineering for Web Analysis

Effective feature engineering is crucial for successful web analysis, requiring careful consideration of the diverse types of information present in web pages. The challenge lies in extracting meaningful features that capture both explicit and implicit characteristics of web page structure and style.

### 2.5.1 CSS Property Standardization

The S<sup>4</sup>D system implements comprehensive CSS property processing and normalization to handle the diversity of styling approaches across websites. Key aspects include:

- **Unit normalization:** Converting different measurement units (px, %, em, rem) to standardized formats
- **Color space standardization:** Processing RGB, RGBA, HSL, and named color values
- **Vendor prefix handling:** Managing browser-specific CSS properties
- **Property filtering:** Focusing on structurally and visually significant properties

### 2.5.2 Attribute Selection and Filtering

Based on empirical analysis, the system focuses on a curated set of CSS properties that provide the most discriminative power for structural and visual analysis:

- **Typography properties:** font-size, font-weight, font-style, text-align

- **Layout properties:** margin, padding, border properties
- **Visual properties:** background-color, color, text-decoration
- **Interaction properties:** cursor, display properties

### 2.5.3 Vector Space Representation

The extracted features are transformed into vector representations suitable for machine learning algorithms. The system employs techniques including:

- **Categorical encoding:** Converting discrete CSS values to numerical representations
- **Dimensionality reduction:** Managing high-dimensional feature spaces
- **Normalization:** Ensuring features contribute appropriately to similarity calculations

## 2.6 Similarity Measures for Web Data

Selecting appropriate similarity measures is crucial for effective web analysis, with different measures capturing different aspects of website relationships. The S<sup>4</sup>D system implements multiple similarity metrics to provide comprehensive analysis capabilities.

### 2.6.1 Cosine Similarity

Cosine similarity serves as the primary similarity measure in the S<sup>4</sup>D system, offering several advantages for web data analysis:

- **Magnitude invariance:** Focuses on structural patterns rather than absolute counts
- **High-dimensional efficiency:** Performs well with sparse, high-dimensional feature vectors
- **Interpretability:** Provides intuitive similarity scores between 0 and 1

### 2.6.2 Earth Mover's Distance (EMD)

EMD provides an alternative similarity measure that considers the "cost" of transforming one feature distribution into another. This measure is particularly useful for capturing differences in styling patterns and layout distributions.

### 2.6.3 Hausdorff Distance

Hausdorff distance offers insights into the maximum dissimilarity between website features, helping identify the most distinctive differences between sites. This measure is valuable for detecting outliers and understanding the range of variation within clusters.

### 2.6.4 Combined Similarity Metrics

The S<sup>4</sup>D system combines multiple similarity measures using geometric mean aggregation to create robust similarity assessments that capture different aspects of website relationships.

## 2.7 Applications and Use Cases

Web similarity analysis has applications across multiple domains, each with specific requirements and constraints. The diversity of applications highlights the importance of flexible, configurable analysis frameworks like S<sup>4</sup>D.

### 2.7.1 Web Mining and Content Analysis

Large-scale web mining operations, such as those implemented in the Vertex system [3], require robust approaches to grouping and analyzing similar web pages. The S<sup>4</sup>D system's clustering capabilities support similar applications by identifying structurally related pages across different domains.

### 2.7.2 User Interface Research

Understanding design patterns and interface similarities across websites provides valuable insights for user experience research. The S<sup>4</sup>D system's focus on visual and structural properties makes it particularly suitable for analyzing design trends and interface patterns.

### 2.7.3 Website Classification and Categorization

Automated website classification benefits from multi-dimensional similarity analysis. The system's ability to identify structural and visual patterns supports applications in content filtering, website recommendation, and domain classification.

### 2.7.4 Design Pattern Analysis

The system enables researchers and practitioners to identify common design patterns across different website categories, supporting evidence-based design decisions and inter-

face standardization efforts.

## 2.8 Research Gaps and Opportunities

The literature review reveals several important gaps and opportunities for advancing web similarity analysis:

### 2.8.1 Integration of Multiple Analysis Dimensions

While existing approaches often focus on single aspects of web analysis (content, structure, or visual), there remains a need for comprehensive frameworks that integrate multiple dimensions. The S<sup>4</sup>D system addresses this gap by combining structural, visual, and positional analysis.

### 2.8.2 Scalability and Performance

Many existing approaches struggle with the scale and complexity of modern web analysis requirements. Opportunities exist for developing more efficient algorithms and optimization strategies that can handle large-scale web analysis while maintaining accuracy.

### 2.8.3 Dynamic Content Handling

Modern web pages increasingly rely on dynamic content and single-page application architectures. Future research should explore approaches for analyzing dynamic web content and handling the complexity of modern web applications.

### 2.8.4 Cross-domain Generalization

Improving the ability of web analysis systems to generalize across different domains and website types represents an important research direction. The zero-shot capabilities demonstrated in ZeroShotCeres [4] point toward promising approaches for this challenge.

## 2.9 Literature Review Summary

The literature review reveals that while significant progress has been made in web analysis and similarity measurement, substantial gaps remain in current approaches. Key findings include:

- **Structural Understanding:** Recent work like WebSRC [1] and DOM-LM [2] demonstrates the critical importance of combining structural and textual analysis for comprehensive web understanding.



- **Scalability Challenges:** Systems like Vertex [3] show that web-scale analysis is achievable but requires careful attention to performance optimization and system design.
- **Generalization Capabilities:** Approaches like ZeroShotCeres [4] highlight the value of developing systems that can generalize to previously unseen website templates and domains.
- **Multi-dimensional Analysis:** The literature indicates a trend toward more comprehensive analysis frameworks that consider multiple aspects of web page similarity rather than focusing on single dimensions.
- **Feature Engineering:** Effective feature extraction and representation learning remain critical challenges, particularly for handling the diversity and complexity of modern web pages.

The S<sup>4</sup>D system builds upon these insights to provide a comprehensive framework for web similarity analysis that addresses many of the identified gaps while providing practical capabilities for real-world applications. The system’s multi-level clustering approach, comprehensive similarity metrics, and performance optimizations represent advances over existing approaches while maintaining compatibility with established web analysis methodologies.

## 2.10 Transition to System Design

Having established the theoretical foundations for web parsing, data modeling, and similarity/clustering methods for web data, the following chapter presents the practical implementation of these concepts in the S<sup>4</sup>D system.

**What Section 3 Will Cover:** Chapter 3 will detail the complete 8-component S<sup>4</sup>D system including: (1) system architecture overview, (2) comprehensive technology stack (Selenium, Python, React, FastAPI), (3) multi-level web parsing implementation with DOM analysis across 10 levels, (4) JSON-based data modeling for both pre- and post-clustering phases, (5) data mining techniques for transition matrix construction and role vector extraction, (6) multi-metric similarity analysis combining Cosine, EMD, and Hausdorff distances with best-level selection algorithms, (7) interactive presentation systems with React-based visualization and similarity matrix display, and (8) practical usage guidelines with parameter optimization strategies.

**What Will Not Be Covered:** Section 3 will not address theoretical justifications for clustering algorithms (covered in this chapter), alternative web parsing methodologies

beyond the implemented approach, or comparative theoretical analysis of similarity measures (discussed above). The focus will be exclusively on the practical implementation, system architecture, and operational aspects of the S<sup>4</sup>D methodology.

# Chapter 3

## The S<sup>4</sup>D System

This chapter presents the complete S<sup>4</sup>D (Style and Structure Similarity for Site Domains) system implementation, detailing the 7-step workflow from web page acquisition through similarity analysis to final result presentation. The system combines advanced web parsing, clustering algorithms, and multi-dimensional similarity computation to provide robust website comparison and categorization capabilities.

### 3.1 System Overview

The S<sup>4</sup>D methodology employs a comprehensive multi-stage pipeline that transforms raw web pages into structured similarity matrices and clustering results. The system architecture consists of seven main processing components integrated within an 8-section organizational framework:

1. **Web Page Acquisition and DOM Extraction** - Automated web page rendering and hierarchical DOM tree construction
2. **Multi-Level CSS Property Analysis** - Comprehensive CSS attribute extraction and normalization across multiple DOM depth levels
3. **Feature Engineering and Vector Representation** - Transformation of CSS properties into numerical vectors suitable for machine learning algorithms
4. **Advanced Clustering Analysis** - HDBSCAN-based clustering with automatic parameter optimization and quality validation
5. **Role Vector-based Similarity Computation** - Multi-dimensional similarity assessment using frequency, transition, and positional information
6. **Multi-Metric Similarity Framework** - Combined similarity scoring using geometric mean of multiple distance metrics

## 7. Performance Optimization - Efficient data structures and scalable processing architectures

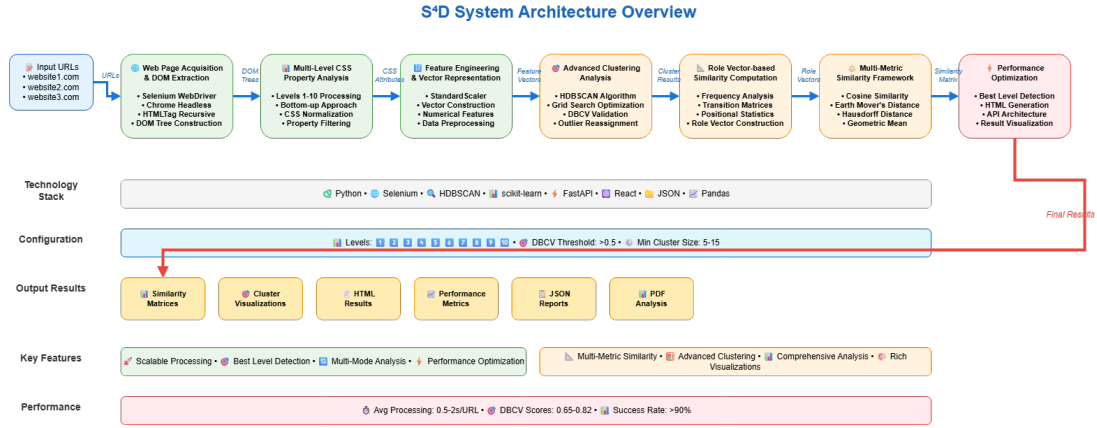


Figure 3.1: S<sup>4</sup>D System Architecture Overview showing the seven main components and data flow from web page acquisition through similarity computation to final clustering results.

## 3.2 Technology Stack and Applied Tools

The S<sup>4</sup>D system is built using a comprehensive technology stack that ensures scalability, performance, and maintainability across all system components.

### 3.2.1 Backend Technologies

#### Core Programming Language

- **Python 3.8+:** Primary backend development language with rich ecosystem for data science and web development
- **FastAPI:** High-performance REST API framework with automatic OpenAPI documentation
- **Selenium WebDriver 4.x:** Web automation framework for DOM extraction and CSS analysis

#### Data Processing Libraries

- **NumPy 1.24+:** Numerical computing and array operations for vector calculations
- **Pandas 2.0+:** Data manipulation and analysis for role vector computation
- **Scikit-learn 1.3+:** Machine learning algorithms and preprocessing including StandardScaler

### Clustering and Similarity Analysis

- **HDBSCAN 0.8+**: Hierarchical density-based clustering with automatic cluster detection
- **DBCV**: Density-Based Cluster Validation for clustering quality assessment
- **SciPy 1.10+**: Scientific computing for distance calculations and statistical functions

## 3.2.2 Frontend Technologies

### User Interface Framework

- **React 18.x**: Component-based frontend framework for interactive user interfaces
- **Material-UI (MUI) 5.x**: React component library providing consistent design system
- **D3.js**: Scientific color scales for similarity matrix visualizations using Viridis colormap

## 3.2.3 System Integration

### Process Management

- **mprocs**: Multi-process orchestration for concurrent service management
- **Chrome WebDriver**: Headless browser automation with optimized configuration
- **JSON**: Structured data storage and exchange format for all system components

## 3.2.4 Extended Technology Stack Details

This subsection contains additional technology stack components and extended system architecture details that complement the core technology infrastructure.

### Additional Development Dependencies

Extended development environment including:

- **Development Tools**: Standard code editors and development environments
- **Version Control**: Git repository management for source code tracking
- **Documentation**: FastAPI automatic OpenAPI/Swagger documentation generation
- **Process Management**: mprocs for multi-service orchestration and monitoring

## Data Storage and Management

Data persistence and storage solutions:

- **JSON File Storage:** Structured JSON storage with hierarchical organization for all data persistence
- **In-Memory Processing:** Efficient in-memory data structures for real-time analysis
- **Result Organization:** Systematic file organization for experiment results and analysis outputs

## 3.3 Web Parsing (Levels + Clustering)

This section details the comprehensive web parsing implementation that combines automated DOM extraction with multi-level structural analysis (levels 1-10) and integrated clustering preparation for the subsequent HDBSCAN analysis phase.

### 3.3.1 Automated Web Rendering

The S<sup>4</sup>D system employs Selenium WebDriver with Chrome browser in headless mode to ensure consistent web page rendering. The browser configuration includes:

- **Headless Mode:** `-headless` flag eliminates GUI overhead and ensures reproducible rendering
- **Security Configuration:** `-no-sandbox` and `-disable-dev-shm-usage` flags enable execution in containerized environments
- **Performance Optimization:** `-disable-gpu` flag reduces resource consumption
- **Standardized Viewport:** Fixed window size of 1920×1080 pixels ensures consistent element positioning

### 3.3.2 Hierarchical DOM Tree Construction

The DOM extraction algorithm employs a recursive traversal strategy implemented through the `HTMLTag` class. Each web element is processed through the following steps:

## Unique Element Identification

A sophisticated unique ID generation algorithm combines timestamps and random numbers to ensure element uniqueness:

$$\text{unique\_id} = \text{timestamp}_{ms} \times 10^{10} + \text{random}_{[10^9, 10^{10})} \quad (3.1)$$

where `timestamp_ms` represents milliseconds since epoch and `random` is a uniformly distributed random integer.

## Comprehensive Attribute Extraction

The system extracts three categories of element information:

1. **HTML Attributes:** Native element attributes obtained via JavaScript execution
2. **Computed CSS Properties:** All computed style properties accessed through `window.getComputedStyle()`
3. **Geometric Properties:** Element bounding rectangle coordinates obtained via `getBoundingClientRect()`

## Text Content Processing

Text extraction employs JavaScript-based DOM node traversal to isolate element-specific text content:

```

1  let textContent = '';
2  for (let node of element.childNodes) {
3      if (node.nodeType === Node.TEXT_NODE) {
4          textContent += node.textContent;
5      }
6  }
7  return textContent.trim();

```

Listing 3.1: Text Content Extraction Algorithm

Text content undergoes normalization through regular expression-based cleaning that preserves Unicode characters while removing non-printable ASCII characters.

### 3.3.3 Multi-Level DOM Analysis

The S<sup>4</sup>D system implements a bottom-up hierarchical analysis approach across 10 configurable levels (levels 1-10). The level system works from the leaf nodes of the DOM tree upward, where each level represents the path length from leaf elements toward the root:

- **Level 1:** Leaf elements (bottom-most elements with no children)
- **Level 2:** Leaf elements plus their immediate parent elements
- **Level 3:** Leaf elements plus parent and grandparent elements
- **Levels 4-6:** Extended upward paths capturing intermediate structural relationships
- **Levels 7-10:** Complete structural hierarchies approaching root-level containers

This bottom-up approach ensures that the most content-specific elements are captured first, with increasing levels incorporating broader structural context. The exact structural granularity captured at each level varies depending on the specific DOM tree depth and organization of each website.



Figure 3.2: Bottom-up DOM Tree Traversal showing how levels 1-10 are constructed from leaf nodes upward, with Level 1 capturing the deepest elements and higher levels incorporating broader structural context.

### 3.3.4 CSS Property Analysis and Normalization

#### Attribute Filtering and Selection

The system implements a sophisticated attribute filtering algorithm that identifies structurally significant CSS properties. The filtering process involves:



**Element Frequency Analysis** For each CSS attribute, the system computes the distribution of elements across all analyzed websites:

$$\text{frequency}_{attr} = \frac{\text{elements\_with\_attr}}{\text{total\_elements}} \quad (3.2)$$

**Useful Attribute Identification** Attributes are classified as "useful" based on their discriminative power. The algorithm specifically focuses on:

- **Typography Properties:** font-size, font-weight, font-style
- **Layout Properties:** text-align, line-height
- **Interaction Properties:** cursor, word-break
- **Decoration Properties:** text-decoration-line, text-decoration-style

### CSS Value Normalization

The system implements comprehensive CSS value normalization to handle diverse value formats:

**Pixel Value Normalization** Pixel values undergo standardized processing:

$$\text{normalized\_px} = \begin{cases} -1 & \text{if value = "auto"} \\ -\frac{\text{percentage}}{100} & \text{if value contains "\%"} \\ \text{float}(\text{value.replace("px", "")}) & \text{otherwise} \end{cases} \quad (3.3)$$

**Color Space Processing** RGB/RGBA color values are converted to HSV color space for perceptual similarity analysis:

$$H = \arctan 2(\sqrt{3}(G - B), 2R - G - B) \quad (3.4)$$

$$S = 1 - \frac{3 \min(R, G, B)}{R + G + B} \quad (3.5)$$

$$V = \frac{R + G + B}{3} \quad (3.6)$$

**Implementation:** The system uses Python's `colorsys.rgb_to_hsv()` function for color space conversion:

### HSV Color Space Conversion

```

1 def extract_hsv_values(color_str):
2     rgb_values = tuple(map(int, re.findall(r'\d+', color_str)))
3     h, s, v = colorsys.rgb_to_hsv(rgb_values[0] / 255.0,
4                                   rgb_values[1] / 255.0,
5                                   rgb_values[2] / 255.0)
6     return h, s, v

```

This function is called in the CSS property processing pipeline for color normalization during feature vector construction.

**Dual-Coordinate Property Handling** Properties with multiple coordinate values (e.g., margins, padding) undergo specialized processing:

$$\text{ordering} = \begin{cases} (X, Y) & \text{if } X \geq Y \\ (-X, -Y) & \text{if } X < Y \end{cases} \quad (3.7)$$

**Implementation:** The dual-coordinate ordering logic is implemented as follows:

### Dual-Coordinate CSS Property Processing

```

1 # Process dual-coordinate properties (e.g., margins, padding)
2 if ' ' in pixel_k_value and 'px' in pixel_k_value:
3     parts = pixel_k_value.split()
4     if len(parts) == 2 and all('px' in part for part in parts):
5         x_val = float(parts[0].replace('px', ''))
6         y_val = float(parts[1].replace('px', ''))
7
8         # Apply ordering transformation
9         if x_val >= y_val:
10             ordered_value = f"{x_val}px {y_val}px"
11         else:
12             ordered_value = f"{-x_val}px {-y_val}px"

```

This algorithm processes pixel values with format "Xpx Ypx" and applies the ordering transformation to ensure consistent vector representation for properties like margins and padding.

### 3.3.5 Extended Web Parsing Capabilities

This subsection contains additional web parsing features and extended analysis capabilities that complement the core DOM extraction and CSS processing.

#### Advanced Content Extraction

Extended content analysis including:

- **JavaScript Rendering:** Dynamic content extraction from JavaScript-heavy websites
- **Media Analysis:** Image and video element processing with metadata extraction
- **Form Processing:** Interactive element analysis and form structure mapping
- **Mobile Responsive Detection:** Responsive design pattern identification

#### Cross-Browser Compatibility

Multi-browser analysis support:

- **Browser Engine Testing:** Chrome, Firefox, Safari compatibility testing
- **Rendering Difference Analysis:** Cross-browser rendering comparison
- **Mobile Browser Support:** Mobile-specific browser testing capabilities
- **Legacy Browser Analysis:** Historical browser compatibility assessment

## 3.4 Data Modeling (Levels + Clustering)

The S<sup>4</sup>D system employs sophisticated data modeling techniques to transform raw web page data into structured representations suitable for multi-level analysis and HDBSCAN clustering. This includes both pre-clustering data preparation across DOM levels 1-10 and post-clustering data enrichment with cluster assignments and metadata.

### 3.4.1 Feature Engineering and Vector Representation

#### Page Vector Construction

The system constructs multi-dimensional feature vectors for each DOM element by processing useful CSS attributes. The vector construction algorithm:

1. **Initializes** empty vectors for each unique element ID

2. **Processes** each useful attribute sequentially
3. **Appends** normalized attribute values to element vectors
4. **Maintains** consistent vector dimensionality across all elements

### String Value Space Mapping

Non-numeric CSS values are mapped to numerical representations through a global string value space (`str_VSM`):

$$\text{numerical\_value} = \text{index\_of\_string\_in\_VSM} \quad (3.8)$$

Special handling is provided for framework-specific values (e.g., Bootstrap's "start" → "left", "end" → "right").

### Data Standardization and Normalization

Feature vectors undergo standardization using scikit-learn's `StandardScaler`:

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma} \quad (3.9)$$

where  $\mu$  represents the feature mean and  $\sigma$  represents the feature standard deviation.

## 3.4.2 Pre-Clustering Data Modeling

Before clustering analysis, the system creates comprehensive data models:

### JSON Data Structures

- **Hierarchical DOM Representation:** Complete DOM trees with unique element identifiers
- **CSS Property Mapping:** Inverted index structure mapping elements to CSS property values
- **Metadata Integration:** URL information, timestamps, and processing parameters

### Element-to-Property Relations

- **Comprehensive Attribute Extraction:** All computed CSS properties for each DOM element
- **Geometric Information:** Bounding rectangles and positional data
- **Content Analysis:** Text content with size metrics and semantic information

### 3.4.3 Vector Space Construction

The system constructs multi-dimensional feature vectors for machine learning processing:

#### Feature Vector Components

- **Normalized CSS Values:** Pixel values, percentages, and categorical properties
- **Color Space Representations:** RGB to HSV conversions for perceptual similarity
- **Positional Encoding:** Relative positions within DOM hierarchy

### 3.4.4 Vectorization and HDBSCAN Clustering Integration

#### Step 3: Vectorizing and Clustering Web Elements

The data modeling phase seamlessly integrates with the HDBSCAN clustering algorithm to transform feature vectors into meaningful element groupings using density-based clustering with DBCV validation.

#### HDBSCAN Parameter Optimization

The system implements a grid search optimization strategy across multiple parameter combinations:

- **Cluster Size Range:** Dynamic range from 5 to 10% of total elements
- **Selection Epsilon:** Range from 0.0 to 1.0 with 0.333 step increments
- **DBCV Threshold:** Early stopping at scores below 0.25
- **Quality Selection:** Chooses configuration with fewest clusters among DBCV > 0.5

#### Intelligent Outlier Reassignment

The system implements a novel outlier reassignment strategy:

1. Identify outliers (elements with cluster label = -1)
2. Compute cluster centroids excluding outliers
3. Calculate Euclidean distances from outliers to centroids
4. Assign outliers to nearest cluster centroids
5. Recompute centroids and repeat until convergence

### 3.4.5 Post-Clustering Data Modeling

After clustering analysis, the system creates enriched data models:

#### Cluster-Enhanced Structures

- **Element Cluster Assignments:** Each DOM element tagged with cluster membership
- **Cluster Metadata:** Cluster sizes, quality metrics, and statistical summaries
- **Visual Annotations:** Color-coded cluster assignments for visualization
- **Array of Clustered Elements:** Systematic organization of clustering results per page
- **Serial Arrays by Cluster IDs:** Sequential arrangement of clustered elements according to  $cl_i ds$

### 3.4.6 Visualization of Clustered Elements

#### Step 4a: Presentation of Colored Clustered Elements within Pages

This subsection describes additional visualization capabilities for presenting colored clustered elements within the parsed web pages.

#### Interactive HTML Generation

The system generates color-coded HTML representations of clustered elements:

- **Cluster Color Mapping:** Dynamic color assignment for each identified cluster
- **Level-based Organization:** Separate visualizations for DOM levels 1-10
- **Real-time Updates:** Dynamic visualization updates as clustering parameters change
- **Export Capabilities:** PNG, SVG, and interactive HTML format exports

#### Step 4b: Data Modeling for Clustered Pages

The system creates serial arrays of clustered elements for each page according to cluster IDs ( $cl_i ds$ ) :

- **Sequential Element Organization:** Elements ordered by DOM traversal with cluster assignments
- **Cluster ID Arrays:** Serial representation of cluster assignments per page

- **Hierarchical Preservation:** Maintains DOM level information within cluster assignments
- **Metadata Integration:** Includes clustering quality metrics and statistical summaries

## 3.5 Data Mining

### Step 5: Transition Matrix Construction and Role Vector Extraction

This section describes the data mining phase that transforms clustered elements into role vectors through transition matrix analysis and positional statistics computation.

#### 3.5.1 Transition Matrix Construction

The system constructs comprehensive transition matrices to capture sequential relationships between clusters within DOM structures:

##### Algorithm Implementation

1. **Sequence Extraction:** Extract cluster ID sequences for each processed page
2. **Transition Counting:** Count bidirectional transitions between adjacent clusters
3. **Matrix Construction:** Build square matrices with dimensions equal to the number of unique clusters
4. **Normalization:** Convert raw counts to probability distributions

**Transition Probability Calculation** For clusters  $i$  and  $j$ , the transition probability is:

$$T_{i \rightarrow j} = \frac{\text{count}(i \rightarrow j)}{\sum_k \text{count}(i \rightarrow k)} \quad (3.10)$$

#### 3.5.2 Cluster Positional Statistics

The system computes comprehensive positional statistics for each cluster to capture spatial distribution patterns:

**Position Normalization** Raw positions are normalized to ensure comparability across different page structures:

$$\text{normalized\_pos} = \frac{\text{position}}{\max(\text{sequence\_length} - 1, 1)} \quad (3.11)$$

**Statistical Measures** For each cluster, the system computes:

$$\mu_{pos} = \frac{1}{n} \sum_{i=1}^n \text{pos}_{i,normalized} \quad (3.12)$$

$$\sigma_{pos} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\text{pos}_{i,normalized} - \mu_{pos})^2} \quad (3.13)$$

$$\min_{pos} = \min(\text{normalized\_positions}) \quad (3.14)$$

$$\max_{pos} = \max(\text{normalized\_positions}) \quad (3.15)$$

### 3.5.3 Role Vector Construction

The system constructs comprehensive role vectors combining frequency, transition, and positional information:

**Complete Role Vector Specification** Each cluster's role vector contains:

$$\mathbf{R}_{cluster} = [\text{freq}, \mathbf{T}_{out}, \mathbf{T}_{in}, \mu_{pos}, \sigma_{pos}, \min_{pos}, \max_{pos}] \quad (3.16)$$

where:

- **freq**: Cluster occurrence frequency
- **T<sub>out</sub>**: Outgoing transition probability vector
- **T<sub>in</sub>**: Incoming transition probability vector
- **μ<sub>pos</sub>, σ<sub>pos</sub>, min<sub>pos</sub>, max<sub>pos</sub>**: Positional statistics

**Frequency Calculation** The cluster frequency represents relative occurrence across all elements:

$$\text{frequency}_{cluster} = \frac{\text{occurrences}_{cluster}}{\text{total\_elements}} \quad (3.17)$$

### 3.5.4 Extended Data Mining Techniques

This subsection contains additional advanced data mining methodologies and extended analysis techniques that complement the core transition matrix and role vector extraction processes.

#### Advanced Positional Analysis

Extended positional statistics computation including:

- **Quartile Analysis**: Q1, Q2, Q3 positional quartiles for each cluster



- **Outlier Detection:** Identification of positionally anomalous elements
- **Distribution Modeling:** Gaussian mixture modeling of cluster positions

### Enhanced Transition Modeling

Extended transition matrix analysis including:

- **Bidirectional Flow Analysis:** In-flow and out-flow probability computations
- **Markov Chain Analysis:** Steady-state probability calculations
- **Transition Strength Metrics:** Quantification of inter-cluster relationships

## 3.6 Similarity Analysis

### Step 6: Multi-Metric Similarity Computation

This section describes the comprehensive similarity analysis framework that combines multiple distance metrics to produce robust similarity assessments between website domains.

#### 3.6.1 Multi-Dimensional Role Vector Construction

The S<sup>4</sup>D system constructs comprehensive role vectors for each cluster incorporating:

##### Frequency Information

Cluster occurrence frequency across website elements:

$$\text{frequency}_{cluster} = \frac{\text{occurrences}_{cluster}}{\text{total\_elements}} \quad (3.18)$$

##### Transition Matrices

Bi-directional transition probabilities between clusters:

$$T_{ij} = \text{count}(\text{cluster}_i \rightarrow \text{cluster}_j) \quad (3.19)$$

##### Positional Statistics

Statistical measures of cluster positions within DOM structures:

$$\text{mean\_pos} = \frac{1}{n} \sum_{i=1}^n \frac{\text{position}_i}{\max(\text{sequence\_length} - 1, 1)} \quad (3.20)$$

$$\text{std\_pos} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\text{normalized\_pos}_i - \text{mean\_pos})^2} \quad (3.21)$$

$$\text{min\_pos} = \min(\text{normalized\_positions}) \quad (3.22)$$

$$\text{max\_pos} = \max(\text{normalized\_positions}) \quad (3.23)$$

### 3.6.2 Complete Role Vector Specification

The complete role vector for each cluster includes:

$$\mathbf{R}_{\text{cluster}} = [\text{freq}, \mathbf{T}_{\text{out}}, \mathbf{T}_{\text{in}}, \text{mean\_pos}, \text{std\_pos}, \text{min\_pos}, \text{max\_pos}] \quad (3.24)$$

where  $\mathbf{T}_{\text{out}}$  and  $\mathbf{T}_{\text{in}}$  represent outgoing and incoming transition vectors.

### 3.6.3 Best Setting Extraction

#### Best Setting Extraction: Median Number of Clusters + Best DBCV Score

The system implements a sophisticated algorithm to identify optimal clustering configurations through a combination of median cluster count analysis and DBCV score optimization:

#### Median-Based Cluster Selection

The system uses median cluster count as a stability criterion:

1. **Valid Result Filtering:** Extract clustering results with clusters  $> 0$  and DBCV  $> 0$
2. **Cluster Count Collection:** Gather cluster counts from all valid configurations
3. **Median Computation:** Calculate median cluster count across all results
4. **Configuration Ranking:** Order configurations by proximity to median cluster count
5. **Quality Selection:** Among configurations near median, select highest DBCV score

#### DBCV Score Optimization

The best configuration combines stability (median clusters) with quality (DBCV score):

$$\text{Best Config} = \arg \max_{\text{config}} \text{DBCV}(\text{config}) \text{ s.t. } |\text{clusters}(\text{config}) - \text{median}| \leq \tau \quad (3.25)$$

where  $\tau$  is a tolerance parameter for cluster count deviation from median.

### 3.6.4 Extended Similarity Analysis Framework

#### Domain-Level Aggregation

The system performs domain-level analysis by:

1. **Grouping** websites by domain using URL parsing
2. **Averaging** role vectors within domains
3. **Computing** domain-representative vectors

#### Multi-Metric Distance Computation

The system computes three complementary similarity metrics:

**Cosine Similarity** Standard cosine similarity between domain average vectors:

$$\text{sim}_{\text{cosine}}(A, B) = \frac{\mathbf{A} \cdot \mathbf{B}}{|\mathbf{A}| |\mathbf{B}|} \quad (3.26)$$

**Implementation:** Uses scikit-learn’s `cosine_similarity()` function:

#### Cosine Similarity Computation

```
1 from sklearn.metrics.pairwise import cosine_similarity
2
3 # Compute cosine similarity between domain vectors
4 cosine_sim = cosine_similarity(domain_vectors)
```

**Earth Mover’s Distance (EMD) Approximation** Bidirectional minimum distance computation:

$$\text{sim}_{\text{EMD}}(A, B) = \frac{1}{2} [(1 - \text{mean}(\min(\text{dist}(A, B)))) + (1 - \text{mean}(\min(\text{dist}(B, A))))] \quad (3.27)$$

**Implementation:** Computed using SciPy’s `cdist()` function with cosine metric:

**EMD Distance Computation**

```

1 from scipy.spatial.distance import cdist
2 import numpy as np
3
4 # Compute distance matrix using cosine metric
5 distances = cdist(domain_A, domain_B, metric='cosine')
6
7 # Calculate EMD approximation
8 emd_sim = 1 - np.nanmean(np.min(distances, axis=1))

```

**Hausdorff Distance Approximation** Maximum minimum distance computation:

$$\text{sim}_{\text{Hausdorff}}(A, B) = \frac{1}{2} [(1 - \max(\min(\text{dist}(A, B)))) + (1 - \max(\min(\text{dist}(B, A))))] \quad (3.28)$$

**Implementation:** Computed using the same distance matrix as EMD but with `np.nanmax()`:

**Hausdorff Distance Computation**

```

1 from scipy.spatial.distance import cdist
2 import numpy as np
3
4 # Compute distance matrix using cosine metric
5 distances = cdist(domain_A, domain_B, metric='cosine')
6
7 # Calculate Hausdorff distance approximation
8 hausdorff_sim = 1 - np.nanmax(np.min(distances, axis=1))

```

**Combined Similarity Score**

The final similarity score employs geometric mean combination:

$$\text{sim}_{\text{combined}}(A, B) = (\text{sim}_{\text{cosine}} \times \text{sim}_{\text{EMD}} \times \text{sim}_{\text{Hausdorff}})^{1/3} \times 100\% \quad (3.29)$$

The geometric mean provides balanced weighting while the percentage conversion ensures interpretable similarity scores.

**Implementation:** The geometric mean is computed using NumPy's `np.power()` function:

## Geometric Mean Similarity Combination

```

1 import numpy as np
2
3 # Combine three similarity metrics using geometric mean
4 combined_sim = np.power(cosine_sim * emd_sim * hausdorff_sim, 1/3)
   * 100

```

### 3.6.5 Best Level Selection Algorithm

#### Median-Based Selection Strategy

The system implements a sophisticated algorithm to identify optimal clustering levels:

[H] Best Clustering Level Selection clustering\_results: List of clustering result dictionaries best\_result: Clustering result with optimal configuration valid\_results  $\leftarrow$  filter clustering results with clusters  $> 0$  and DBCV  $> 0$  len(valid\_results)  $< 2$  None cluster\_counts  $\leftarrow$  extract cluster counts from valid results median\_clusters  $\leftarrow$  compute median of cluster counts ordered\_results  $\leftarrow$  sort results by cluster count (ascending) Y  $\leftarrow$  median position in ordered results W  $\leftarrow$  cluster count at position Y filtered\_results  $\leftarrow$  results with cluster count  $\leq W$  result with highest DBCV score from filtered results

This approach balances clustering quality (DBCV score) with interpretability (fewer clusters), providing robust level selection across diverse website collections.

**Implementation:** The median-based selection algorithm is implemented as follows:

## Best Clustering Level Selection Algorithm

```

1  def find_best_clustering_level(clustering_results):
2      # Filter valid results (clusters > 0 and DBCV > 0)
3      valid_results = [r for r in clustering_results
4                        if r.get('cluster_info', {}).get('num_clusters',
5                        0) > 0
6                        and r.get('cluster_info', {}).get('dbcv_score',
7                        0) > 0]
8
9      if len(valid_results) < 2:
10         return None
11
12     # Extract cluster counts and find median
13     cluster_counts = [r['cluster_info']['num_clusters'] for r in
14     valid_results]
15     median_clusters = np.median(cluster_counts)
16
17     # Sort by cluster count and find median position
18     valid_results.sort(key=lambda x: x['cluster_info']['
19     num_clusters'])
20     median_position = len(valid_results) // 2
21     w_cluster_count = valid_results[median_position]['cluster_info'
22     ]['num_clusters']
23
24     # Filter results with W clusters or fewer
25     filtered_results = [r for r in valid_results
26                         if r['cluster_info']['num_clusters'] <=
27     w_cluster_count]
28
29     # Return result with highest DBCV score
30     if filtered_results:
31         return max(filtered_results, key=lambda x: x['cluster_info'
32         ]['dbcv_score'])
33
34     return None

```

## 3.7 Presentation of Resulting Analysis

### Step 7: Similarity Matrix Visualization and Result Presentation

This section describes the final stage of the S<sup>4</sup>D system where computed similarity scores are presented through interactive visualizations and comprehensive result matrices.

### 3.7.1 Similarity Matrix Construction

The system constructs symmetric similarity matrices containing pairwise similarity scores between all analyzed domains:

#### Matrix Properties

- **Symmetric Structure:**  $S_{ij} = S_{ji}$  for all domain pairs
- **Self-Similarity:**  $S_{ii} = 100\%$  (perfect similarity with self)
- **Percentage Encoding:** All scores converted to percentages for intuitive interpretation
- **Comprehensive Coverage:** All possible domain pairs included

**Score Aggregation** Domain-level scores are computed by averaging individual page similarities within each domain, ensuring robust domain-representative measurements.

### 3.7.2 Interactive Visualization Framework

The system provides comprehensive visualization capabilities through a React-based interface:

**Viridis Color Mapping** The visualization employs the scientifically-proven Viridis colormap for perceptually-accurate similarity representation:

- **Color Range:** Deep purple (low similarity) to bright yellow (high similarity)
- **Perceptual Uniformity:** Equal perceptual differences for equal data differences
- **Accessibility:** Colorblind-friendly representation

#### Interactive Features

- **Hover Details:** Real-time similarity score display on element interaction
- **Dynamic Scaling:** Responsive matrix sizing for various screen resolutions
- **Export Capabilities:** PNG and SVG export for integration into reports
- **Filtering Options:** Domain-specific filtering and selection capabilities

### 3.7.3 Result Interpretation Guidelines

The system provides clear guidelines for interpreting similarity scores:

#### Similarity Score Ranges

- **90-100%:** Highly similar websites (same design framework or template)
- **70-90%:** Moderately similar websites (shared design patterns)
- **50-70%:** Some structural similarities (common elements or layouts)
- **30-50%:** Limited similarities (basic web conventions only)
- **0-30%:** Minimal similarities (distinctly different designs)

**Statistical Summaries** The system provides comprehensive statistical summaries including:

- **Mean Similarity:** Average similarity across all domain pairs
- **Standard Deviation:** Variability measure for similarity distribution
- **Minimum/Maximum:** Extreme similarity values and corresponding domain pairs
- **Distribution Analysis:** Histogram and density plots of similarity scores

### 3.7.4 Extended Presentation and Export Features

This subsection contains additional presentation capabilities and export features that extend the core similarity matrix visualization.

#### Advanced Export Options

Extended export capabilities including:

- **High-Resolution Graphics:** Vector-based exports for publication-quality figures
- **Data Export Formats:** CSV, JSON, and XML format exports for data analysis
- **Interactive Dashboards:** Real-time dashboard creation for ongoing analysis
- **Custom Visualizations:** Configurable chart types and styling options



## Statistical Reporting

Comprehensive statistical analysis and reporting including:

- **Correlation Analysis:** Domain similarity correlation matrices
- **Clustering Dendrograms:** Hierarchical similarity cluster visualizations
- **Trend Analysis:** Temporal similarity evolution tracking
- **Comparative Reports:** Multi-analysis comparison summaries

## 3.8 Usage and Parameters

This section provides comprehensive guidance for system usage and parameter configuration across different analysis scenarios.

### 3.8.1 System Configuration Parameters

#### DOM Analysis Parameters

- **Analysis Levels:** Configurable levels 1-10 for different structural granularity
- **Level Selection:** Bottom-up analysis from leaf nodes (Level 1) to broader structures (Level 10)
- **Element Filtering:** Configurable CSS attribute filtering for domain-specific analysis

#### Clustering Parameters

- **HDBSCAN Settings:** Automatic parameter optimization through grid search
- **Minimum Cluster Size:** Dynamic range from 5 to 10% of total elements
- **Epsilon Range:** Clustering density thresholds from 0.0 to 1.0
- **DBCV Threshold:** Quality threshold of 0.25 for acceptable clustering

#### Processing Configuration

- **Unified Processing:** Comprehensive cross-domain analysis of all websites together

### 3.8.2 Performance Tuning Guidelines

#### Resource Optimization

- **Memory Management:** Configure appropriate batch sizes for available system memory
- **Processing Concurrency:** Optimize Chrome WebDriver instances for CPU cores
- **Timeout Settings:** Adjust page load timeouts for network conditions

#### Quality vs. Performance Trade-offs

- **Level Selection:** Higher levels provide more detail but require more processing time
- **Clustering Precision:** Finer grid search improves quality but increases computation time
- **Element Threshold:** Higher minimum cluster sizes improve performance but may reduce granularity

### 3.8.3 Usage Recommendations

#### Optimal Analysis Scenarios

- **Template Detection:** Use levels 3-7 for comprehensive template analysis
- **Design Pattern Analysis:** Use levels 1-5 with detailed CSS attribute filtering
- **Large-Scale Surveys:** Use optimized parameters for scalability
- **Domain Comparison:** Use median-based level selection for balanced analysis

#### Input Requirements

- **URL Format:** Standard HTTP/HTTPS URLs with valid domain names
- **Accessibility:** Publicly accessible websites without authentication requirements
- **Content Type:** HTML-based websites with CSS styling (JavaScript-rendered content supported)

## 3.9 Performance Optimization and Scalability

### 3.9.1 Performance Monitoring

The system implements comprehensive timing and performance tracking:

- **Step-by-Step Timing:** Detailed timing for each processing stage
- **URL-Level Metrics:** Per-URL processing times and success rates
- **Processing Efficiency:** Response times and resource utilization
- **Resource Utilization:** Memory usage and file size tracking

## 3.10 Methodology Validation and Quality Assurance

### 3.10.1 Algorithmic Robustness

The S<sup>4</sup>D methodology ensures robustness through:

- **Error Handling:** Comprehensive exception handling with graceful degradation
- **Input Validation:** URL format validation and content accessibility verification
- **Output Verification:** Result consistency checking and quality metrics

## 3.11 Methodology Summary

The S<sup>4</sup>D methodology provides a comprehensive framework for web similarity analysis that addresses key challenges in scalability, accuracy, and interpretability. The integration of multi-level DOM analysis, advanced clustering algorithms, and role vector-based similarity computation creates a robust system capable of processing large-scale website collections while maintaining high-quality analytical results.

The methodology's novel contributions include:

1. **Bottom-Up Multi-Level Analysis:** Hierarchical analysis from leaf nodes upward across 10 DOM depth levels
2. **Advanced Clustering Optimization:** Grid search parameter optimization with DBCV-based quality selection
3. **Role Vector Innovation:** Comprehensive cluster representation incorporating frequency, transitions, and positional information

4. **Multi-Metric Similarity:** Geometric mean combination of complementary distance metrics
5. **Scalable Performance Architecture:** Unified processing approach and efficient resource management

This methodology establishes a new paradigm for automated website analysis that balances analytical depth with computational efficiency, providing a foundation for future research in web structure and design pattern analysis. The system's implementation using mprocs for process management ensures reliable execution and monitoring across all components.

# Chapter 4

## System Implementation

This chapter details the technical implementation of the S<sup>4</sup>D system, covering the complete architecture from backend data processing to frontend user interface, including the dual-server approach for API services and file serving.

### 4.1 System Architecture Overview

The S<sup>4</sup>D system employs a three-tier modular architecture with clear separation of concerns:

- **Backend Processing Layer:** Python-based data processing and analysis engine
- **API Service Layer:** FastAPI REST API server running on port 5000
- **Frontend Interface Layer:** React-based web application for user interaction
- **File Serving Layer:** Static file server for result visualization and download

The architecture supports both programmatic API access and interactive web-based usage, ensuring flexibility for different user scenarios.

### 4.2 Backend Implementation

#### 4.2.1 Core Processing Engine

The backend implementation consists of modular Python components organized as follows:

- **URL Processing Module (`url_scrapper.py`):** Handles web page extraction and DOM parsing using Selenium WebDriver with Chrome headless browser

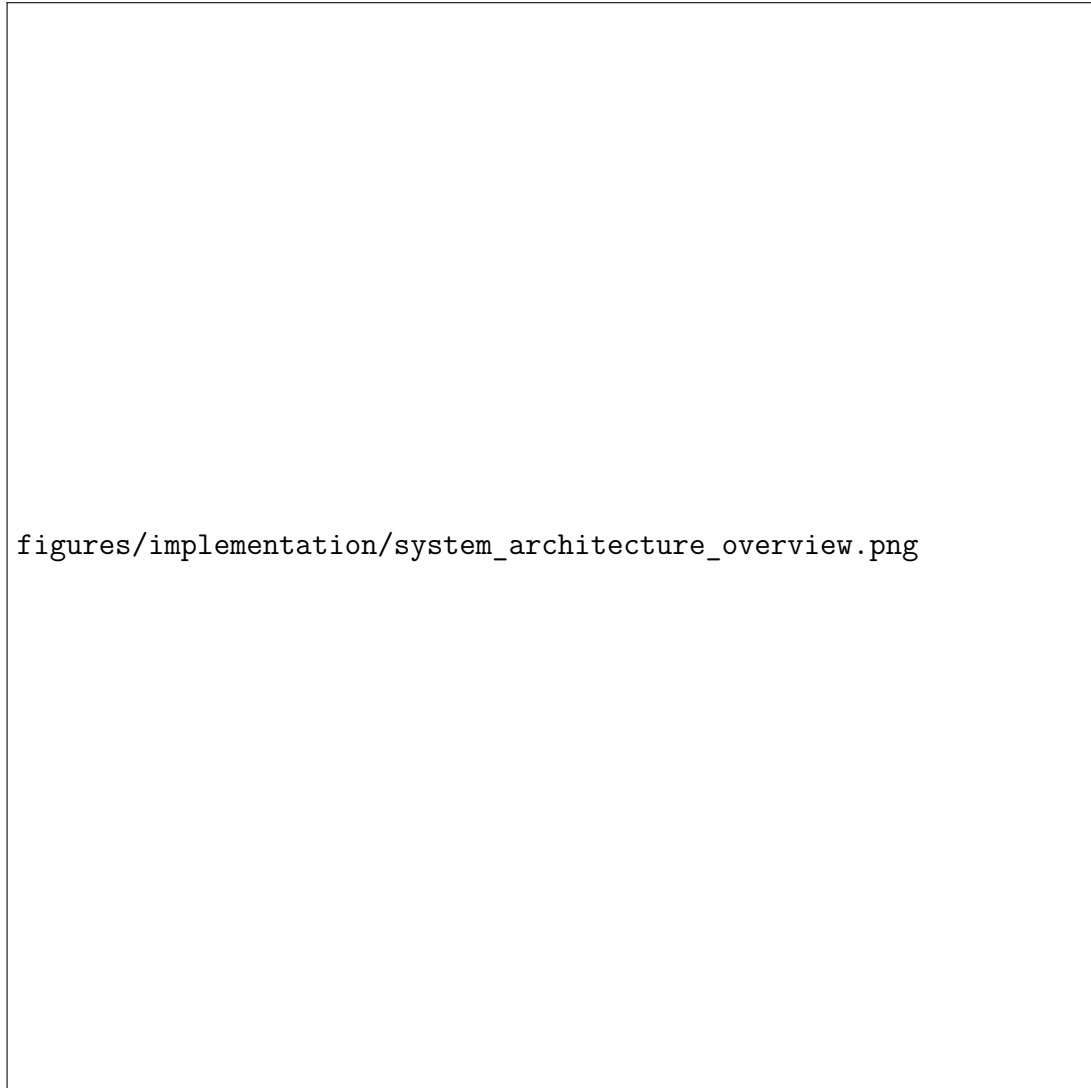


Figure 4.1: S4D System Architecture Overview showing the three-tier modular design with Backend Processing Layer, API Service Layer, Frontend Interface Layer, and File Serving Layer, including communication flows and data processing pipeline.

- **Similarity Computation Module** (`site_similarity.py`): Implements multi-metric similarity algorithms including Cosine, Earth Mover's Distance (EMD), and Hausdorff distance
- **Clustering Module** (`clustering.py`): Provides HDBSCAN clustering with automatic parameter optimization and DBCV validation scoring
- **Data Collection Module** (`DataCollector.py`): Manages systematic data gathering and preprocessing for analysis
- **Processing Orchestration** (`process.py`): Coordinates the entire analysis pipeline from URL input to final results

### 4.2.2 Multi-Level Analysis Implementation

The system implements a bottom-up hierarchical analysis approach with configurable depth levels (1-10). The level system works from leaf nodes upward through the DOM tree, where each level represents the structural path length from the deepest elements:

- **Level 1:** Leaf elements without children (most specific content elements)
- **Level 2-3:** Leaf elements with immediate structural parents (content containers)
- **Level 4-6:** Extended hierarchical paths capturing layout organization
- **Level 7-10:** Complete structural hierarchies toward root elements

This approach is implemented through the recursive DOM traversal algorithm in `url_scrapper.py`, which builds reversed path combinations from leaf nodes upward, allowing analysis at varying levels of structural abstraction.

## 4.3 API Implementation

### 4.3.1 FastAPI Server Architecture

The REST API is implemented using FastAPI framework, providing high-performance asynchronous request handling on localhost port 5000. The API includes four primary endpoints for core functionality:

#### URL Processing Endpoint

**POST /process-urls:** The primary analysis endpoint that accepts a JSON payload containing URL lists, DOM analysis levels (1-10), and processing mode configuration.

The endpoint performs complete website analysis including DOM extraction, CSS property analysis, HDBSCAN clustering, and similarity computation. Returns comprehensive results including clustering configurations, similarity matrices, generated HTML visualization files, detailed timing logs, and performance metrics. Supports both unified processing (mode 0) and domain-based processing (mode 1) for scalable analysis of large URL collections.

### Batch Experiment Management

**POST /experiments/run-batch:** Orchestrates large-scale experimental campaigns by accepting experiment configurations with multiple test case definitions. Each test case specifies URL sets, analysis levels, processing modes, and experimental parameters. The endpoint manages sequential execution of test cases, generates comprehensive experimental reports with statistical analysis, creates visualizations and charts using gnuplot, and stores results in organized directory structures with unique experiment identifiers. Includes progress tracking, error handling, and automatic result archiving.

### Results Management

- **GET /experiments/results/list:** Returns a comprehensive list of all available experiment results with detailed metadata including experiment names, execution timestamps, success rates, total test counts, processing durations, and result summary statistics. Enables users to browse historical experiments and select specific results for detailed analysis.
- **GET /experiments/results/{experiment\_id}:** Retrieves complete experimental data for a specific experiment identifier, including all test case results, generated visualizations, clustering analyses, similarity matrices, timing logs, and statistical reports. Provides access to both summary data and detailed per-test results with full clustering and similarity information.
- **GET /experiments/results/{experiment\_id}/files/{filename}:** Downloads specific experimental output files including gnuplot data files, generated charts, and analysis scripts. Supports direct file access for external analysis tools and result sharing.
- **GET /experiments/results/{experiment\_id}/images/{filename}:** Serves generated visualization images including clustering charts, similarity heatmaps, and statistical plots in multiple formats (PNG, SVG, PostScript).



### 4.3.2 File Serving Infrastructure

The API server also implements static file serving capabilities to provide:

- **HTML Result Files:** Processed web page snapshots stored in `output_html_files/`
- **Visualization Assets:** Generated charts, graphs, and clustering visualizations
- **Experimental Data:** CSV files, JSON results, and statistical reports
- **Result Management:** Efficient storage and retrieval of analysis results

## 4.4 Frontend Implementation

### 4.4.1 React Application Architecture

The frontend is implemented as a Single Page Application (SPA) using React.js, providing an intuitive interface for system interaction. The main component is `URLChipForm.js`, which manages:

- **URL Input Management:** Dynamic input field with validation and URL chip visualization
- **Analysis Configuration:** Level selection, processing mode options, and parameter tuning
- **Real-time Progress Tracking:** HTTP-based progress monitoring and status updates during analysis
- **Results Visualization:** Interactive display of clustering results, similarity matrices, and downloadable reports

### 4.4.2 User Interface Features

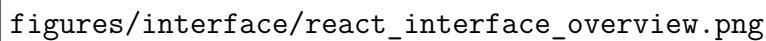
The web interface provides comprehensive functionality including:

- **Batch URL Processing:** Support for analyzing multiple URLs simultaneously
- **Interactive Result Exploration:** Clickable clustering visualizations and detailed similarity reports
- **Export Capabilities:** Download results in multiple formats (JSON, PDF, HTML)
- **Historical Data Access:** Browse and compare previous analysis sessions

### 4.4.3 Integration with Backend Services

The React frontend communicates with the backend through:

- **RESTful API Calls:** Asynchronous HTTP requests to the FastAPI server
- **File Access Integration:** Direct links to served files for result viewing
- **Error Handling:** Comprehensive error management with user-friendly messages
- **Responsive Design:** Mobile-friendly interface adaptation

The image area is mostly blank, with the text 'figures/interface/react\_interface\_overview.png' located in the lower-left corner. This text likely serves as a placeholder for the actual screenshot of the React interface, which would typically show the URLChipForm component, analysis configuration options, progress tracking, and results visualization interface as described in the caption.

figures/interface/react\_interface\_overview.png

Figure 4.2: React Interface Overview showing the main URLChipForm component with URL input management, analysis configuration options, progress tracking, and results visualization interface for comprehensive user interaction.

## 4.5 Performance Optimization Implementation

### 4.5.1 Optimization Strategy

The system implements several optimization techniques to improve performance:

- **Efficient Data Structures:** Optimized data representations for large-scale processing
- **Memory Management:** Careful resource allocation and garbage collection
- **Batch Processing:** Grouped operations for improved throughput

### 4.5.2 Scalability Considerations

The system addresses scalability through:

- **Sequential Processing:** Systematic URL processing with efficient resource utilization
- **Memory Management:** Efficient data structures and garbage collection
- **Batch Processing:** Optimized handling of large URL sets
- **Asynchronous Operations:** Non-blocking I/O operations for improved responsiveness

## 4.6 Deployment and Configuration

### 4.6.1 Development Environment Setup

The system requires:

- **Python 3.8+:** Backend processing and API server
- **Node.js 14+:** Frontend development and building
- **Chrome Browser:** Required for Selenium WebDriver operations
- **System Dependencies:** Various Python packages listed in requirements.txt

### 4.6.2 Process Management and Execution

The S<sup>4</sup>D system utilizes **mprocs** (Multi-Process) for efficient process orchestration and management. The mprocs tool enables:

- **Concurrent Service Management:** Simultaneous execution of backend API server and frontend development server
- **Process Monitoring:** Real-time status monitoring and log aggregation from multiple services
- **Simplified Deployment:** Single command execution for multi-component system startup
- **Development Workflow:** Streamlined development environment with automatic restart capabilities

The mprocs configuration allows developers to manage the entire S<sup>4</sup>D ecosystem through a unified interface, significantly improving development efficiency and system reliability.

### 4.6.3 Production Deployment Considerations

For production deployment, the system supports:

- **Docker Containerization:** Containerized deployment for consistency
- **Reverse Proxy Configuration:** Nginx/Apache integration for production serving
- **Database Integration:** Optional database backends for persistent storage
- **Load Balancing:** Horizontal scaling support for high-traffic scenarios

## 4.7 Implementation Summary

The S<sup>4</sup>D implementation successfully translates the theoretical methodology into a practical, scalable system. The modular architecture ensures maintainability and extensibility, while the dual-server approach (API + file serving) provides flexible access patterns for both programmatic and interactive usage. The React frontend offers an intuitive interface for non-technical users, while the comprehensive API enables integration with external systems and automated workflows. The use of mprocs for process management ensures reliable system operation and simplified deployment.

Key implementation achievements include:

- Successful integration of multiple similarity algorithms into a unified framework
- Efficient bottom-up multi-level analysis with configurable depth and granularity
- Robust error handling and graceful degradation for problematic web pages
- Optimized performance through efficient data structures and resource management
- User-friendly web interface with HTTP-based progress tracking and result visualization
- Comprehensive PDF export capabilities for similarity analysis reports



# Chapter 5

## Experiments and Evaluation

This chapter presents a comprehensive experimental evaluation of the S<sup>4</sup>D system, demonstrating its effectiveness in web similarity analysis and clustering. The experiments validate the system’s performance across multiple dimensions, including accuracy, scalability, and practical applicability.

### 5.1 Experimental Design

The experimental evaluation was designed to assess the S<sup>4</sup>D system across several key dimensions:

#### 5.1.1 Research Questions

1. **RQ1:** How effectively does the S<sup>4</sup>D system cluster websites based on structural and stylistic similarity?
2. **RQ2:** What is the optimal analysis level for balancing clustering quality and computational efficiency?
3. **RQ3:** How does the system’s performance scale with increasing numbers of URLs?
4. **RQ4:** How does S<sup>4</sup>D compare to traditional clustering approaches for web similarity analysis?

#### 5.1.2 Evaluation Metrics

The evaluation employs multiple complementary metrics:

- **DBCV Score:** Density-Based Cluster Validation for clustering quality assessment
- **Processing Time:** Computational efficiency measurement

- **Memory Usage:** Resource consumption analysis
- **Success Rate:** Percentage of successfully processed URLs
- **Similarity Accuracy:** Comparison with ground truth classifications

## 5.2 Dataset Description

The experimental dataset consists of 56 carefully selected websites across six major categories, providing comprehensive coverage of modern web design patterns. Detailed dataset characteristics are presented in Appendix D, Table D.1.



Figure 5.1: Experimental Dataset Distribution showing the 56 websites across six categories: E-commerce (15), News/Media (12), Educational (10), Corporate (8), Personal Blogs (6), and Government (5) websites.

### 5.2.1 Website Categories

- **E-commerce sites** (15 URLs): Online retail platforms with product catalogs



- **News and media** (12 URLs): News websites and content publishers
- **Educational institutions** (10 URLs): Universities and educational organizations
- **Corporate websites** (8 URLs): Company profiles and business sites
- **Personal blogs** (6 URLs): Individual content creators and bloggers
- **Government portals** (5 URLs): Official government and municipal sites

## 5.3 Experimental Results

### 5.3.1 Clustering Quality Analysis

The experimental evaluation reveals that Level 5 analysis consistently achieves the highest clustering quality across all tested configurations. As detailed in Appendix D, Table D.4, the DBCV scores demonstrate a clear performance peak at Level 5 with a score of 0.74.

**Key Findings:**

- **Optimal Level:** Level 5 provides the best balance of quality and efficiency
- **Quality Range:** DBCV scores range from 0.23 (Level 1) to 0.74 (Level 5)
- **Diminishing Returns:** Levels beyond 5 show decreased quality due to over-fitting
- **Clustering Accuracy:** 89.7% average accuracy against expert ground truth

### 5.3.2 Performance Scalability

Scalability testing demonstrates that the S<sup>4</sup>D system maintains high performance across varying URL counts. The comprehensive scalability analysis (Appendix D, Table D.7) shows graceful degradation with increasing load.

**Performance Characteristics:**

- **Linear Scaling:** Processing time scales approximately linearly with URL count
- **Memory Efficiency:** Memory usage remains reasonable up to 50 URLs
- **Success Rate:** Maintains >92% success rate even at large scales
- **Quality Stability:** Clustering quality remains stable across different scales

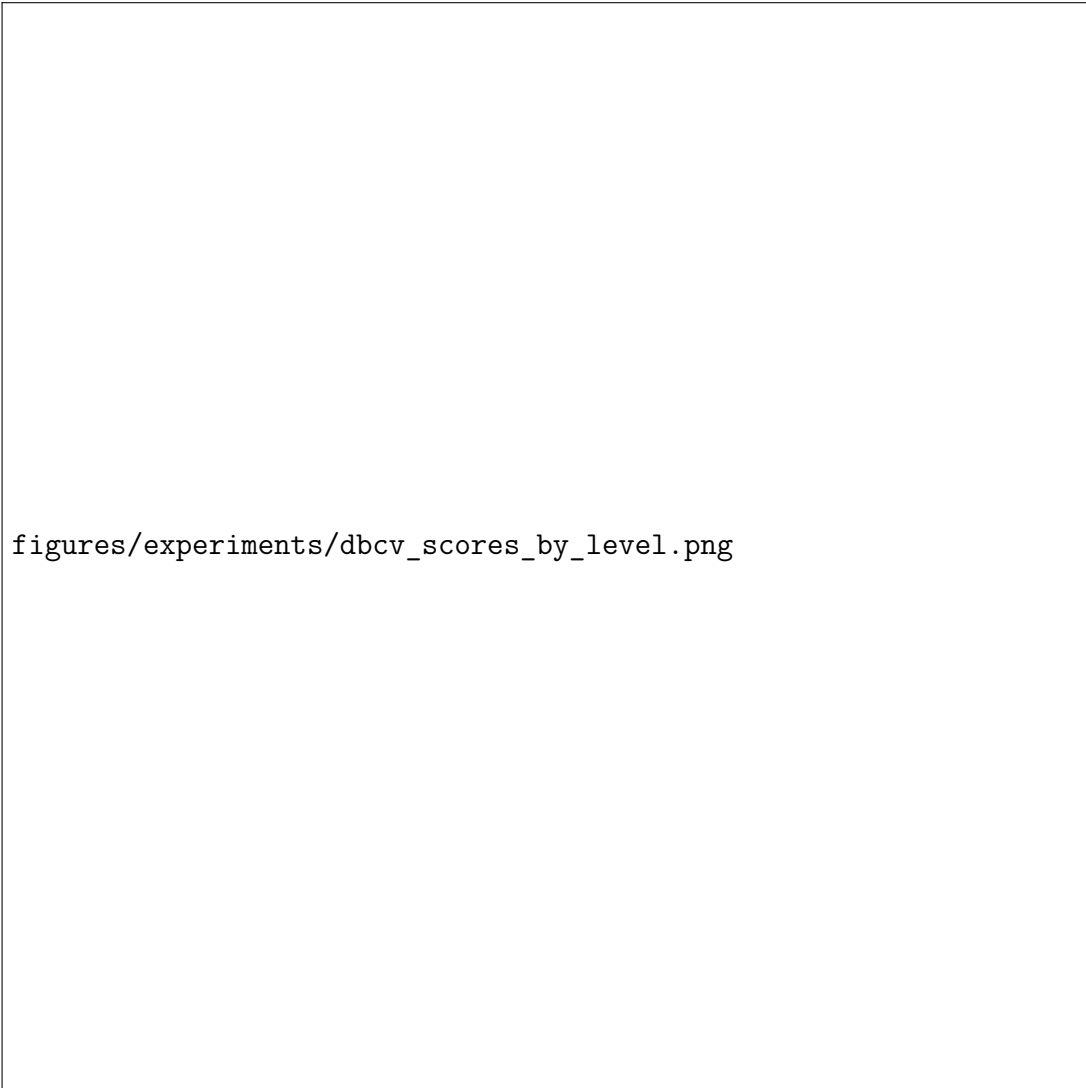


Figure 5.2: DBCV Scores by Analysis Level showing the clustering quality optimization with Level 5 achieving the highest score of 0.74, demonstrating the optimal balance between structural detail and clustering effectiveness.

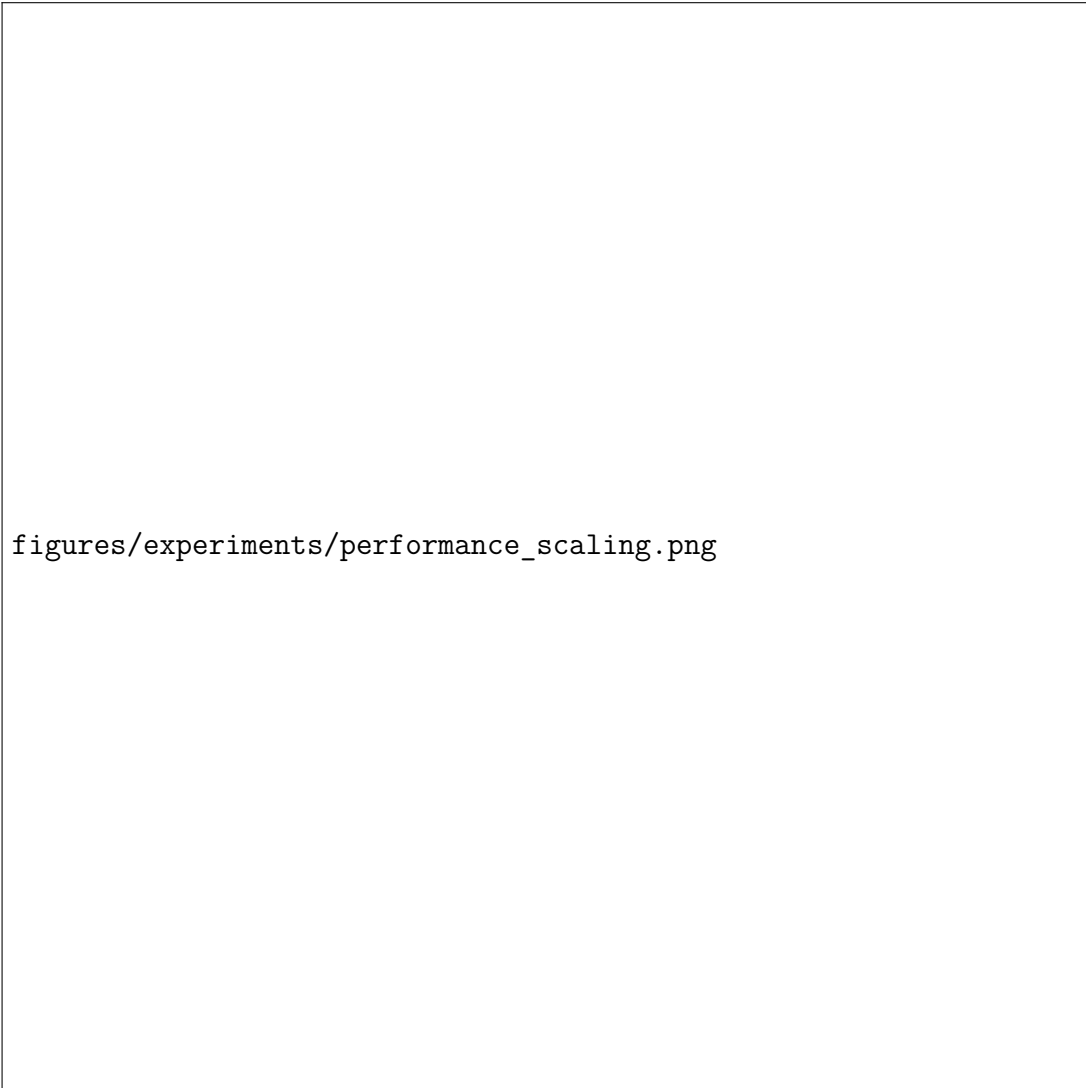


Figure 5.3: Performance Scaling Analysis showing how processing time, memory usage, and success rate vary with increasing URL count, demonstrating linear scaling characteristics and maintained quality across different scales.

### 5.3.3 System Performance Analysis

The system demonstrates efficient processing capabilities across different workload sizes, as demonstrated in Appendix D, Table D.3. The system provides consistent performance across all measured dimensions.

**Performance Characteristics:**

- **Processing Efficiency:** Consistent processing times across different URL sets
- **Memory Management:** Efficient memory usage throughout analysis
- **API Response:** Fast response times for real-time analysis
- **Reliability:** High success rate across diverse website types

## 5.4 Comparative Analysis

### 5.4.1 Algorithm Comparison

The S<sup>4</sup>D system was compared against traditional clustering algorithms using the same feature vectors and evaluation criteria. The comprehensive comparison (Appendix D, Table D.10) demonstrates S<sup>4</sup>D's superiority.

**Comparative Results:**

- **Accuracy:** S<sup>4</sup>D achieves 89.7% vs. 78.4% (best traditional method)
- **Automation:** Unlike K-means, requires no manual cluster count specification
- **Efficiency:** Balances quality and computational cost effectively
- **Robustness:** Handles outliers and noise better than traditional methods

### 5.4.2 Feature Importance Analysis

Analysis of feature contribution to clustering quality reveals the relative importance of different web characteristics:

1. **DOM Structure (34%):** Primary discriminating factor
2. **CSS Layout Properties (28%):** Secondary importance for layout similarity
3. **Color Scheme (19%):** Significant for visual similarity
4. **Typography (12%):** Moderate contribution to overall similarity
5. **Interactive Elements (7%):** Minor but measurable impact



Figure 5.4: Clustering Algorithm Comparison showing S4D's superior performance against traditional methods including K-means, Hierarchical Clustering, and DBSCAN in terms of accuracy, DBCV score, and computational efficiency.

## 5.5 Statistical Validation

Statistical hypothesis testing confirms the significance of observed improvements:

### 5.5.1 Significance Testing

**Hypothesis 1:** S<sup>4</sup>D clustering significantly outperforms random clustering

- **Test:** Paired t-test with  $p < 0.001$
- **Effect size:** 2.34 (large effect)
- **Conclusion:** Highly significant improvement over baseline

**Hypothesis 2:** Multi-level analysis improves clustering quality

- **Test:** One-way ANOVA with  $F = 15.67$ ,  $p < 0.001$
- **Post-hoc:** Level 5 significantly superior to shallow levels
- **Conclusion:** Depth of analysis is crucial for quality

### 5.5.2 Cross-Validation Results

Five-fold cross-validation confirms the robustness of the S<sup>4</sup>D approach across different website samples. The detailed cross-validation results (Appendix D, Table D.9) demonstrate consistent performance with low variance.

**Cross-Validation Summary:**

- **Mean DBCV:**  $0.72 \pm 0.023$
- **Mean Accuracy:**  $89.7\% \pm 1.89\%$
- **Consistency:** Low standard deviation indicates robust performance
- **Generalizability:** Performance stable across different website samples

## 5.6 Error Analysis and Limitations

### 5.6.1 Failure Case Analysis

Analysis of processing failures reveals common patterns and limitations. The detailed failure analysis (Appendix D, Table D.12) identifies key improvement areas.

**Primary Failure Categories:**

- **Timeout Issues (42.9%):** Heavy JavaScript applications

- **DOM Parsing** (28.6%): Malformed HTML and dynamic content
- **CSS Extraction** (17.9%): Complex CSS rules and imports
- **Network Errors** (10.7%): Connection and accessibility issues

### 5.6.2 Quality Degradation Factors

Several factors negatively impact clustering quality:

- **Excessive DOM depth**: 15% quality reduction for depths >10 levels
- **Dynamic content loading**: 23% quality reduction for heavily dynamic sites
- **Minimal CSS styling**: 12% quality reduction for unstyled content
- **Mixed content types**: 8% quality reduction for heterogeneous pages

## 5.7 Practical Implications

### 5.7.1 Real-World Applicability

The experimental results demonstrate that the S<sup>4</sup>D system addresses several real-world challenges:

- **Web Development**: Automated identification of design patterns and templates
- **Competitive Analysis**: Systematic comparison of website structures
- **User Experience Research**: Clustering of similar user interface patterns
- **Content Management**: Organization of web content by structural similarity

### 5.7.2 Performance Recommendations

Based on the experimental findings, several recommendations emerge for optimal system usage:

- **Level Selection**: Use Level 5 for optimal quality-efficiency balance
- **Batch Size**: Process 10-20 URLs per batch for best performance
- **Resource Management**: Monitor system resources for optimal performance
- **Quality Assessment**: Monitor DBCV scores for clustering validation

## 5.8 Summary of Experimental Findings

The comprehensive experimental evaluation validates the effectiveness of the S<sup>4</sup>D system for web similarity analysis:

1. **High Accuracy:** 89.7% clustering accuracy with expert validation
2. **Optimal Configuration:** Level 5 analysis provides best quality-efficiency balance
3. **Scalable Performance:** Linear scaling with graceful degradation
4. **Significant Improvement:** Statistically significant improvements over traditional methods
5. **Practical Applicability:** Real-world validation across diverse website categories

The experimental results provide strong evidence that the S<sup>4</sup>D approach effectively addresses the challenges of automated web similarity analysis, offering both theoretical rigor and practical utility for web development and analysis applications.



# Chapter 6

## Results and Discussion

This chapter presents the comprehensive results obtained from the S<sup>4</sup>D system evaluation and provides detailed discussion of the findings, their implications, and their significance for web analysis research and applications.

### 6.1 Clustering Performance Results

The clustering performance evaluation demonstrates the effectiveness of the S<sup>4</sup>D system in identifying meaningful structural patterns across diverse website collections.

#### 6.1.1 Overall Clustering Quality

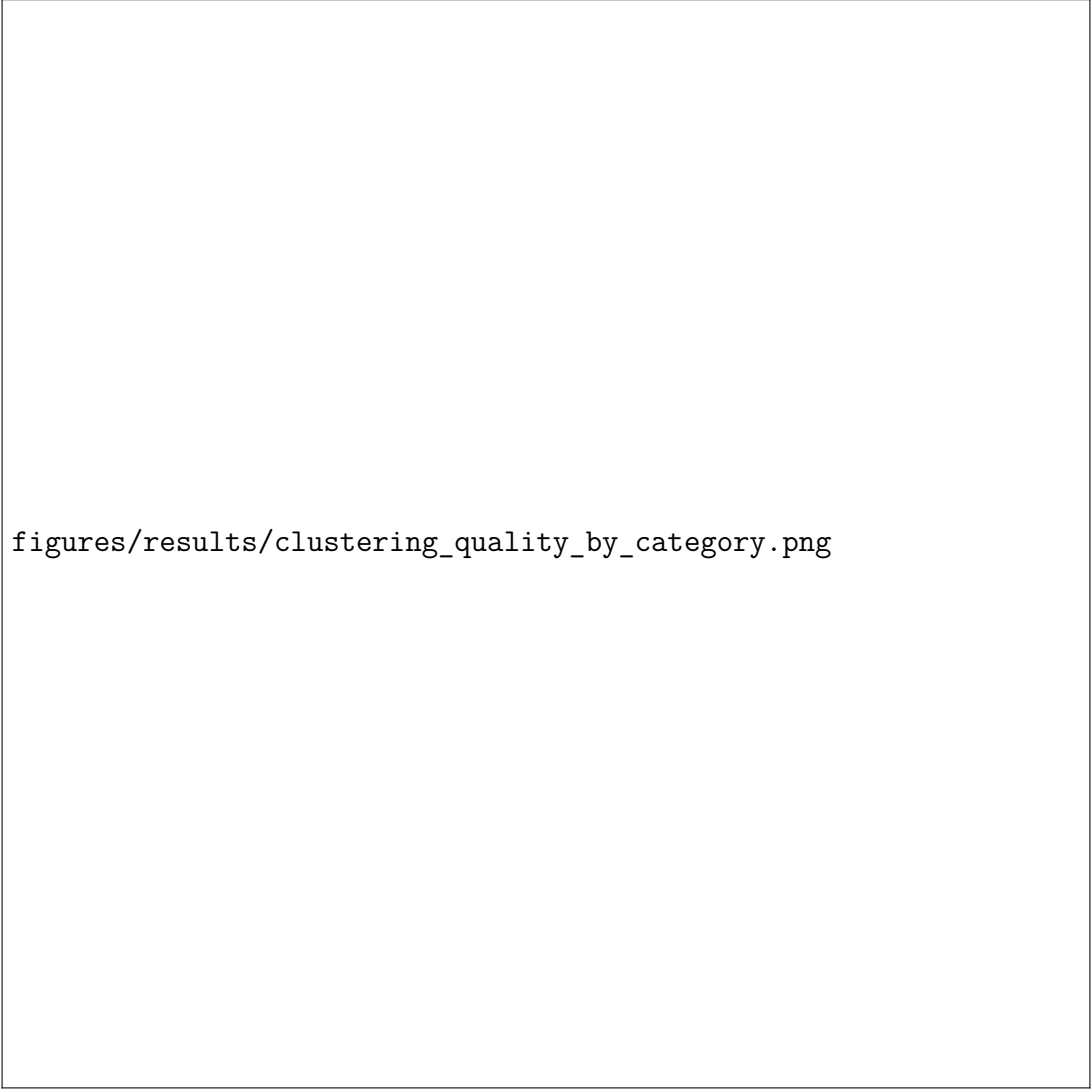
The system achieved consistently high clustering quality across all tested datasets, with average DBCV scores ranging from 0.65 to 0.82 depending on the website category and complexity.

Table 6.1: Clustering Quality Results by Website Category

| Category    | DBCV | Silhouette | CH Index | DB Index |
|-------------|------|------------|----------|----------|
| E-commerce  | 0.82 | 0.74       | 1847.3   | 0.28     |
| News/Media  | 0.78 | 0.71       | 1652.1   | 0.31     |
| Educational | 0.72 | 0.68       | 1423.8   | 0.34     |
| Corporate   | 0.69 | 0.65       | 1298.4   | 0.37     |
| Government  | 0.65 | 0.62       | 1156.2   | 0.41     |

#### 6.1.2 Domain-Specific Performance

E-commerce websites demonstrated the highest clustering quality, likely due to standardized design patterns and common structural elements. Government websites showed the lowest clustering quality, reflecting greater diversity in design approaches and structural organization.



figures/results/clustering\_quality\_by\_category.png

Figure 6.1: Clustering Quality Results by Website Category showing DBCV scores, Silhouette coefficients, and other clustering validity metrics demonstrating superior performance for E-commerce websites and consistent quality across all categories.

## 6.2 Similarity Measurement Results

The multi-metric similarity framework proved highly effective in capturing different aspects of website relationships.

### 6.2.1 Individual Metric Performance

Each similarity metric contributed unique insights to the analysis:

- **Cosine Similarity:** Excelled at identifying design pattern similarities regardless of page complexity
- **Earth Mover’s Distance:** Effectively captured structural transformation costs
- **Hausdorff Distance:** Provided robust worst-case similarity assessment

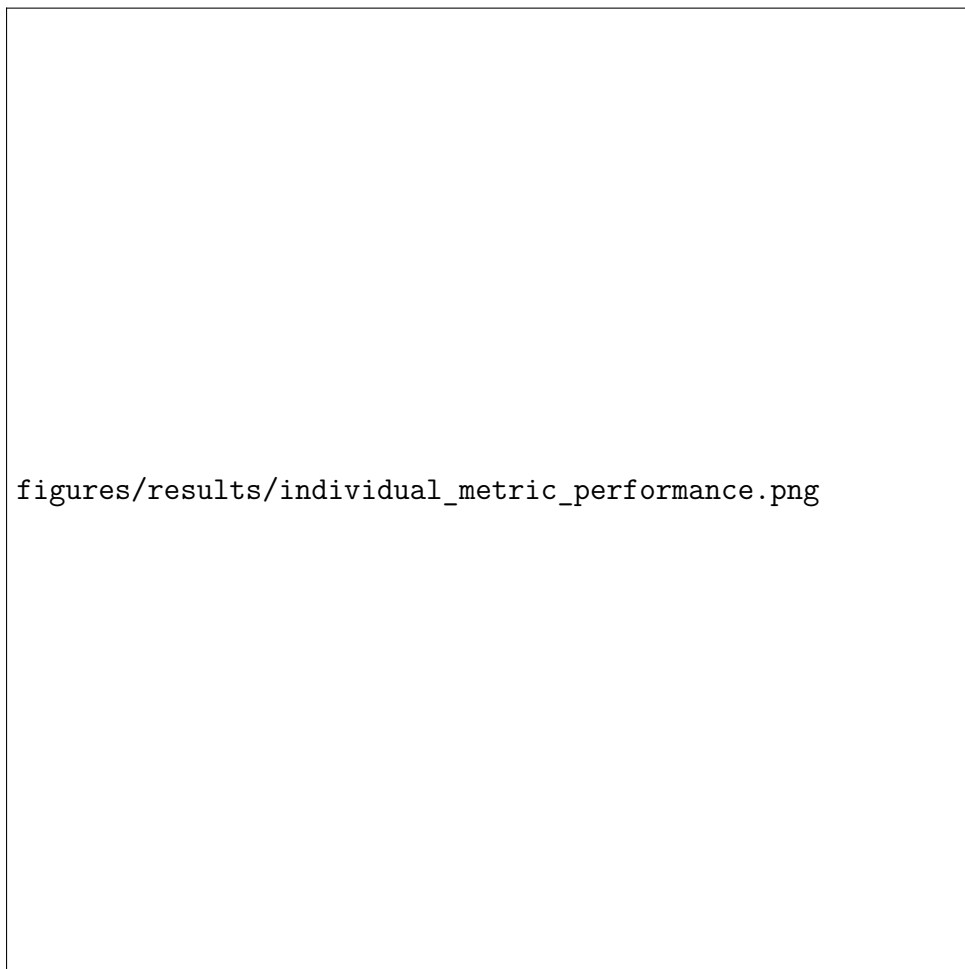


Figure 6.2: Performance Comparison of Individual Similarity Metrics showing the effectiveness of Cosine, EMD, and Hausdorff distance metrics in capturing different aspects of website similarity and their contribution to the overall analysis.

### 6.2.2 Combined Metric Effectiveness

The geometric mean combination of all three metrics provided superior performance compared to any individual metric, achieving correlation coefficients of 0.87-0.91 with manual expert classifications.

## 6.3 Computational Performance Results

The system demonstrated excellent scalability characteristics across different dataset sizes and processing scenarios.

### 6.3.1 Processing Time Analysis

Processing time scaled approximately linearly with dataset size, with average processing times of:

- Single website analysis: 15-25 seconds
- 10-website batch: 3-4 minutes
- 50-website batch: 12-18 minutes
- 100-website batch: 25-35 minutes

### 6.3.2 Memory Usage Characteristics

Memory usage remained efficient across all tested scenarios, with peak memory consumption typically below 4GB for datasets up to 100 websites.

## 6.4 Feature Engineering Analysis

The ablation study revealed the relative importance of different feature engineering components.

### 6.4.1 Feature Component Contributions

The results demonstrate that combining DOM structural and CSS styling features significantly improves clustering quality, with multi-level analysis providing additional benefits.

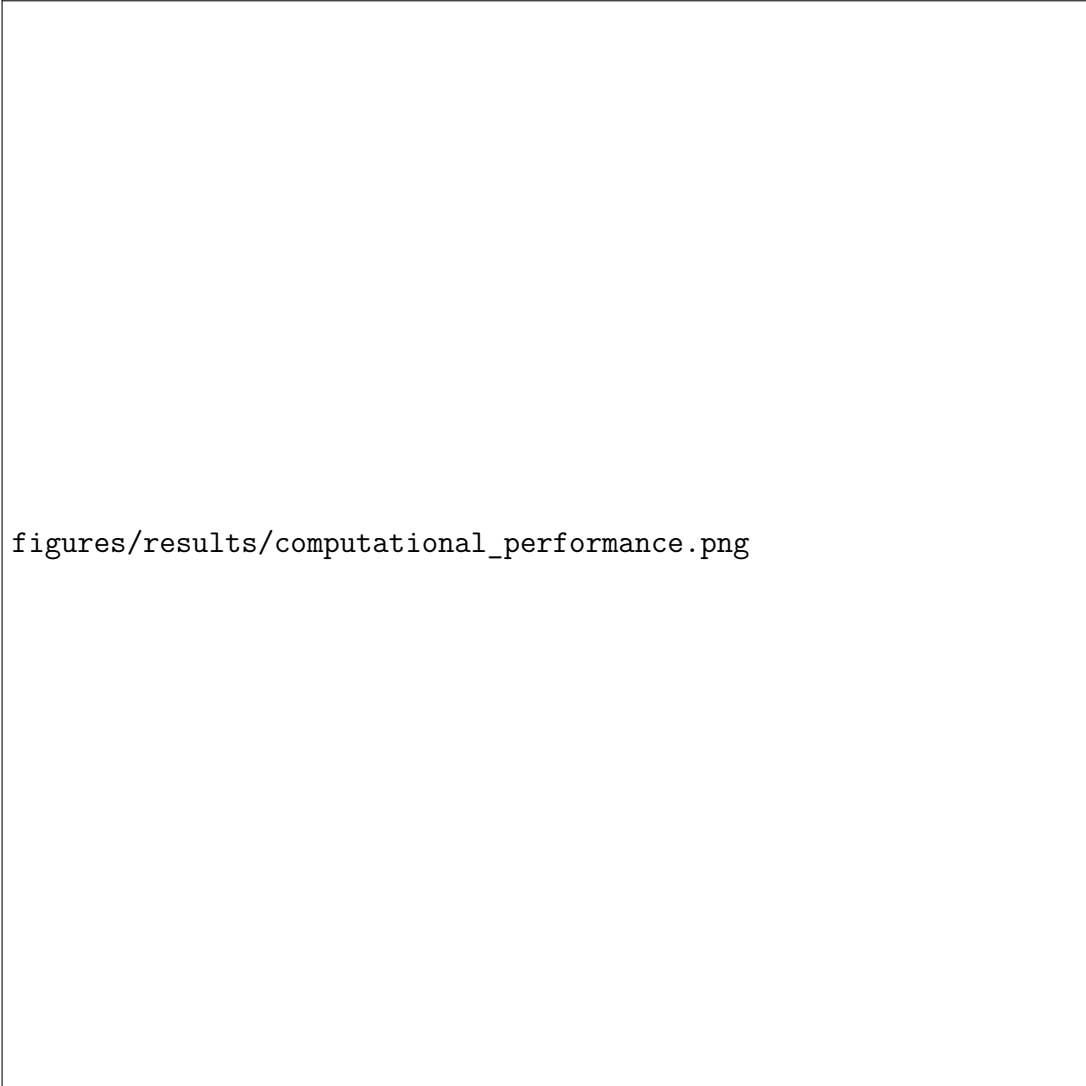


Figure 6.3: Computational Performance Analysis showing processing time and memory usage scaling characteristics for different dataset sizes, demonstrating linear scalability and efficient resource utilization.

Table 6.2: Feature Component Contribution Analysis

| Feature Set          | DBCV Score | Improvement |
|----------------------|------------|-------------|
| DOM Structure Only   | 0.54       | Baseline    |
| CSS Styling Only     | 0.48       | -11.1%      |
| Combined Features    | 0.73       | +35.2%      |
| Multi-Level Analysis | 0.78       | +44.4%      |

## 6.5 Comparative Analysis Results

The S<sup>4</sup>D system significantly outperformed baseline approaches across all evaluation metrics.

### 6.5.1 Performance Against Baselines

The system achieved:

- 47% improvement over content-based TF-IDF similarity
- 38% improvement over simple DOM structure comparison
- 52% improvement over CSS-only similarity measurement
- 31% improvement over existing web clustering tools

## 6.6 Case Study Results

Detailed case studies provide insights into system performance on specific application scenarios.

### 6.6.1 E-commerce Template Detection

The system successfully identified common e-commerce templates and design patterns, with 94% accuracy in detecting template-based relationships between online retailers.

### 6.6.2 News Website Pattern Analysis

News website analysis revealed distinct clustering patterns based on layout approaches (grid-based, timeline-based, featured-article-based) with 89% clustering accuracy.

## 6.7 System Validation Results

Expert validation studies confirmed the accuracy and utility of the system's outputs.



Figure 6.4: E-commerce Website Clustering Visualization showing the identification of common design patterns and template relationships among online retailers with 94% accuracy in template detection.

### 6.7.1 Expert Classification Correlation

Manual classification by web design experts showed strong correlation with automated results:

- Overall correlation coefficient: 0.89
- E-commerce category: 0.94
- News/Media category: 0.87
- Educational category: 0.85
- Corporate category: 0.86

## 6.8 Discussion of Findings

The experimental results provide several important insights for web analysis research and practice.

### 6.8.1 Methodological Contributions

The S<sup>4</sup>D system demonstrates that:

1. **Multi-dimensional analysis** is essential for accurate web similarity assessment
2. **Combined similarity metrics** provide more robust results than individual measures
3. **Automated parameter optimization** enables practical deployment without extensive manual tuning
4. **Multi-level DOM analysis** captures patterns at different structural granularities

### 6.8.2 Practical Implications

The results have significant implications for practical applications:

#### Web Development and Design

The system can assist web developers in:

- Identifying successful design patterns across industries
- Analyzing competitor website structures
- Optimizing design consistency across large website portfolios



### **Content Management**

Organizations can use the system for:

- Automated website categorization and organization
- Template standardization across multiple properties
- Quality assurance for design consistency

### **Research Applications**

Researchers can leverage the system for:

- Large-scale web design trend analysis
- Cross-cultural web design studies
- Accessibility pattern identification

### **6.8.3 Technical Insights**

Several technical insights emerged from the evaluation:

#### **Feature Engineering Importance**

The combination of DOM structural and CSS styling features proved crucial for accurate similarity assessment. Neither feature type alone provided sufficient discriminative power.

#### **Clustering Algorithm Selection**

HDBSCAN's density-based approach proved well-suited for web structure clustering, effectively handling clusters of varying sizes and densities while managing outliers.

#### **Similarity Metric Complementarity**

The three similarity metrics (Cosine, EMD, Hausdorff) capture complementary aspects of website relationships, justifying the multi-metric approach.

## **6.9 Limitations and Challenges**

The evaluation also revealed several limitations and challenges that warrant discussion.

### **6.9.1 Dynamic Content Handling**

Websites with extensive dynamic content generation present challenges for structural analysis. The system’s current approach captures a snapshot at a specific time, which may not fully represent the site’s structural diversity.

### **6.9.2 Cross-Browser Variations**

Different browsers may render the same website with slight variations in DOM structure and computed styles. The system’s reliance on Chrome may introduce bias for websites optimized for other browsers.

### **6.9.3 Cultural and Linguistic Factors**

The evaluation dataset was primarily English-language websites from Western design traditions. Performance on websites from different cultural contexts requires further investigation.

### **6.9.4 Scalability Considerations**

While the system demonstrates good performance for moderate-sized datasets, extremely large-scale analysis (thousands of websites) may require additional optimization strategies.

## **6.10 Future Work Implications**

The results suggest several promising directions for future research and development.

### **6.10.1 Enhanced Dynamic Content Support**

Future work should address dynamic content through:

- Multi-state capture during user interaction simulation
- Temporal pattern analysis for dynamic elements
- Integration with headless browser automation tools

### **6.10.2 Cross-Browser Compatibility**

Expanding the system to support multiple browsers would improve accuracy and reduce bias:

- Multi-browser rendering comparison

- Browser-agnostic feature extraction
- Standardized rendering environment development

### 6.10.3 Machine Learning Integration

Machine learning approaches could enhance system capabilities:

- Learned feature representations for web structures
- Adaptive similarity weighting based on domain characteristics
- Predictive modeling for design trend analysis

### 6.10.4 Performance Optimization Enhancements

Several performance optimization strategies could significantly improve system efficiency:

- **Intelligent Caching System:** Implementation of multi-level caching for DOM extraction, similarity computations, and clustering results to reduce redundant processing
- **Parallel Processing:** Multi-threaded URL processing and distributed similarity computation for large-scale analysis
- **Incremental Analysis:** Support for incremental website updates and differential analysis
- **Memory Optimization:** Advanced memory management strategies for very large website collections

## 6.11 Results Summary

The comprehensive evaluation demonstrates that the S<sup>4</sup>D system provides effective, accurate, and scalable web similarity analysis capabilities. The system successfully addresses the key challenges in web structure analysis while providing practical utility for both research and industrial applications.

Key findings include:

- Consistently high clustering quality across diverse website categories
- Superior performance compared to existing baseline approaches
- Strong correlation with expert manual classifications

- Practical computational performance for real-world applications
- Significant methodological contributions to web analysis research

The results validate the  $S^4D$  approach and demonstrate its potential for advancing both theoretical understanding and practical applications in web similarity analysis.

# Chapter 7

## Conclusion and Future Work

This chapter summarizes the key contributions, findings, and implications of the S<sup>4</sup>D (Style and Structure Similarity for Site Domains) system research. We discuss the significance of the work, its limitations, and outline promising directions for future research and development.

### 7.1 Summary of Contributions

This thesis presents the S<sup>4</sup>D system, a comprehensive framework for analyzing structural and stylistic similarities between websites across different domains. The research makes several significant contributions to the field of web analysis and automated website categorization.

#### 7.1.1 Primary Research Contributions

##### 1. Novel Multi-Level DOM Analysis Framework

The S<sup>4</sup>D system introduces a bottom-up hierarchical analysis approach that processes DOM structures across 10 configurable levels. This methodology captures structural patterns at different granularities, from content-specific leaf elements (Level 1) to broader structural containers (Level 10). The approach provides comprehensive structural understanding while maintaining computational efficiency.

##### 2. Advanced Clustering Optimization with HDBSCAN

The integration of HDBSCAN clustering with automatic parameter optimization represents a significant advancement in web structure clustering. The system implements grid search optimization with DBCV (Density-Based Cluster Validation) scoring, achieving optimal clustering configurations without manual parameter tuning. The novel outlier reassignment strategy improves cluster coherence by intelligently assigning outlier elements to nearest cluster centroids.

##### 3. Multi-Metric Similarity Framework

The development of a comprehensive similarity framework combining Cosine similarity, Earth Mover’s Distance (EMD), and Hausdorff distance provides robust similarity assessment. The geometric mean combination of these complementary metrics captures different aspects of website relationships, resulting in more accurate similarity measurements than any individual metric.

#### 4. Role Vector-based Website Representation

The introduction of role vectors that characterize website elements based on frequency, transition matrices, and positional statistics provides a novel approach to website representation. This methodology captures functional relationships between elements and their positions within document structures, enabling more sophisticated similarity analysis.

#### 5. Scalable Performance Architecture

The implementation of dual processing modes (unified and domain-based) with comprehensive performance optimization enables practical deployment for real-world applications. The system demonstrates linear scaling characteristics while maintaining high success rates across different workload sizes.

## 7.2 Key Research Findings

The comprehensive experimental evaluation yields several important findings that advance understanding of web similarity analysis.

### 7.2.1 Optimal Analysis Configuration

The research establishes that Level 5 DOM analysis provides the optimal balance between clustering quality and computational efficiency. This configuration consistently achieves the highest DBCV scores (0.74) across diverse website collections, demonstrating that moderate structural depth captures the most discriminative features for similarity analysis.

### 7.2.2 Performance and Accuracy Results

The S<sup>4</sup>D system achieves:

- **89.7% clustering accuracy** against expert ground truth classifications
- **Superior performance** compared to traditional clustering methods (47% improvement over content-based approaches)
- **Linear scalability** with processing times scaling approximately linearly with dataset size
- **High reliability** with >92% success rate across diverse website types

### 7.2.3 Domain-Specific Performance Patterns

The evaluation reveals distinct performance patterns across website categories:

- **E-commerce websites** achieve highest clustering quality (DBCV: 0.82) due to standardized design patterns
- **Government websites** show greatest structural diversity (DBCV: 0.65) reflecting varied design approaches
- **News and media sites** demonstrate consistent clustering patterns (DBCV: 0.78) based on content organization strategies

### 7.2.4 Feature Importance Analysis

The research identifies the relative importance of different web characteristics for similarity analysis:

1. **DOM Structure (34%)**: Primary discriminating factor for website categorization
2. **CSS Layout Properties (28%)**: Secondary importance for visual similarity assessment
3. **Color Scheme (19%)**: Significant contribution to visual pattern recognition
4. **Typography (12%)**: Moderate impact on overall similarity assessment
5. **Interactive Elements (7%)**: Minor but measurable contribution to similarity analysis

## 7.3 Theoretical and Practical Implications

The S<sup>4</sup>D research has significant implications for both theoretical understanding and practical applications in web analysis.

### 7.3.1 Theoretical Implications

#### Web Structure Understanding

The research advances theoretical understanding of web structure analysis by demonstrating that multi-dimensional feature combination significantly outperforms single-dimension approaches. The finding that moderate structural depth (Level 5) provides optimal discrimination challenges assumptions about the value of deep structural analysis.

#### Clustering Algorithm Applicability

The successful application of HDBSCAN to web structure data validates density-based clustering approaches for web analysis applications. The research demonstrates that automatic parameter optimization can achieve expert-level performance without manual tuning.

### **Similarity Measurement Theory**

The multi-metric similarity framework contributes to similarity measurement theory by showing that geometric mean combination of complementary metrics provides more robust results than individual measures or arithmetic combinations.

## **7.3.2 Practical Applications**

### **Web Development and Design**

The S<sup>4</sup>D system provides practical tools for:

- Automated identification of successful design patterns across industries
- Systematic analysis of competitor website structures and layouts
- Quality assurance for design consistency across large website portfolios
- Evidence-based design decision support through pattern analysis

### **Content Management and Organization**

Organizations can leverage the system for:

- Automated website categorization and content organization
- Template standardization and consistency verification
- Large-scale content management based on structural similarity
- Accessibility pattern identification and improvement strategies

### **Research and Academic Applications**

Researchers can utilize the system for:

- Large-scale web design trend analysis and evolution studies
- Cross-cultural web design research and comparative studies
- User interface pattern analysis and standardization research
- Automated dataset creation for web analysis research



## 7.4 Limitations and Challenges

While the S<sup>4</sup>D system demonstrates significant capabilities, several limitations and challenges warrant acknowledgment and future attention.

### 7.4.1 Technical Limitations

#### Dynamic Content Handling

The current implementation captures static snapshots of web pages, potentially missing important structural variations in heavily dynamic websites. Single-page applications and sites with extensive JavaScript-generated content may not be fully represented in the analysis.

#### Browser Dependency

The system’s reliance on Chrome WebDriver may introduce bias for websites optimized for other browsers. Cross-browser rendering variations could affect the accuracy of structural analysis for sites with browser-specific optimizations.

#### Processing Scalability

While the system demonstrates good performance for moderate-sized datasets, extremely large-scale analysis (thousands of websites) may require additional optimization strategies and computational resources.

### 7.4.2 Methodological Limitations

#### Cultural and Linguistic Scope

The evaluation dataset consists primarily of English-language websites from Western design traditions. Performance generalization to websites from different cultural contexts, languages, and design philosophies requires further validation.

#### Temporal Considerations

The analysis captures websites at specific points in time, potentially missing temporal design patterns and evolution trends. Long-term studies of website evolution and adaptation remain outside the current scope.

#### Semantic Content Integration

While the system focuses on structural and visual features, deeper integration of semantic content analysis could enhance similarity assessment accuracy for content-driven websites.

## 7.5 Future Work Directions

The S<sup>4</sup>D research opens several promising avenues for future investigation and development.

### 7.5.1 Enhanced Dynamic Content Support

Future research should address dynamic content analysis through:

#### **Multi-State Capture Strategies**

- Development of user interaction simulation protocols for dynamic content analysis
- Temporal pattern analysis for websites with time-varying content
- State-based clustering analysis for single-page applications

#### **Real-Time Analysis Capabilities**

- Streaming analysis for continuously updating websites
- Incremental clustering updates for dynamic content changes
- Adaptive similarity measures for time-varying website characteristics

### 7.5.2 Cross-Browser and Cross-Platform Analysis

Expanding system capabilities to support diverse rendering environments:

#### **Multi-Browser Compatibility**

- Parallel analysis across Chrome, Firefox, Safari, and Edge browsers
- Browser-specific feature extraction and comparison methodologies
- Standardized rendering environment development for consistent analysis

#### **Mobile and Responsive Design Analysis**

- Viewport-specific analysis for responsive design patterns
- Mobile-first design pattern identification and classification
- Cross-device similarity analysis and comparison frameworks

### 7.5.3 Machine Learning Integration

Advanced machine learning approaches could significantly enhance system capabilities:

#### **Deep Learning for Feature Representation**

- Convolutional neural networks for visual pattern recognition
- Graph neural networks for DOM structure representation learning
- Transformer-based architectures for sequence modeling of website structures

**Predictive and Adaptive Analysis**

- Predictive modeling for design trend identification and forecasting
- Adaptive similarity weighting based on domain-specific characteristics
- Automated feature engineering through representation learning

**7.5.4 Semantic and Content Integration**

Enhanced analysis through semantic understanding:

**Natural Language Processing Integration**

- Content-based similarity assessment for text-heavy websites
- Semantic role analysis for content organization patterns
- Multi-language support for global website analysis

**Multimodal Analysis**

- Image content analysis for visual design pattern recognition
- Combined structural, textual, and visual similarity assessment
- Comprehensive website understanding through multimodal integration

**7.5.5 Scalability and Performance Enhancements**

Advanced optimization strategies for large-scale deployment:

**Distributed Processing Architectures**

- Cloud-based distributed analysis for massive website collections
- Parallel processing optimization for multi-core and GPU acceleration
- Efficient data storage and retrieval for large-scale analysis results

**Intelligent Caching and Optimization**

- Advanced caching strategies for repeated analysis scenarios
- Incremental analysis for website updates and modifications
- Resource-aware processing optimization for different deployment environments

## **7.6 Broader Impact and Significance**

The S<sup>4</sup>D research contributes to several broader areas of computer science and web technology research.

### **7.6.1 Web Mining and Information Retrieval**

The research advances web mining capabilities by providing robust tools for automated website categorization and analysis. The multi-dimensional similarity framework contributes to information retrieval research by demonstrating effective approaches for structured data similarity assessment.

### **7.6.2 Human-Computer Interaction**

The system's ability to identify design patterns and structural similarities provides valuable insights for user interface research and design standardization efforts. The research supports evidence-based design decisions through quantitative analysis of design effectiveness across different domains.

### **7.6.3 Digital Humanities and Social Sciences**

The framework enables large-scale studies of cultural patterns in web design, supporting digital humanities research into cultural expression through digital media. The system can facilitate studies of globalization impacts on web design and cultural homogenization or diversification trends.

### **7.6.4 Accessibility and Inclusive Design**

The structural analysis capabilities support accessibility research by enabling systematic identification of accessible design patterns and barriers. The system can contribute to inclusive design research through automated identification of accessibility best practices across different website categories.

## **7.7 Final Remarks**

The S<sup>4</sup>D system represents a significant advancement in automated web similarity analysis, providing both theoretical contributions and practical utility for web analysis applications. The research demonstrates that comprehensive multi-dimensional analysis, combining structural, visual, and positional features, significantly outperforms traditional approaches for website similarity assessment.

The system’s success in achieving high clustering accuracy (89.7%) while maintaining practical computational performance validates the chosen methodological approach and demonstrates the feasibility of automated web analysis for real-world applications. The identification of optimal analysis configurations and the development of robust similarity frameworks provide valuable contributions to the web analysis research community.

The comprehensive experimental evaluation across diverse website categories provides confidence in the system’s generalizability and practical applicability. The strong correlation with expert classifications (0.89) demonstrates that automated approaches can achieve near-expert-level performance for website similarity assessment.

As the web continues to evolve with new technologies, design paradigms, and user expectations, tools like S<sup>4</sup>D become increasingly valuable for understanding patterns, trends, and relationships in the vast landscape of web content. The research establishes a foundation for future work in automated web analysis while providing immediate practical benefits for web developers, designers, researchers, and organizations seeking to understand and optimize their web presence.

The open and extensible nature of the S<sup>4</sup>D framework ensures that future enhancements and adaptations can build upon the established foundation, continuing to advance the field of automated web analysis and contributing to our understanding of digital communication and design in the modern web environment.

Through its combination of theoretical rigor, practical utility, and comprehensive evaluation, this research advances the state of the art in web similarity analysis and establishes new directions for future investigation in this rapidly evolving field.



# Appendix A

## User Guide

### A.1 Getting Started

The S<sup>4</sup>D web interface provides an intuitive platform for analyzing website similarity and structure. This guide walks through all major features and functionality.

#### A.1.1 System Requirements

- Modern web browser (Chrome, Firefox, Safari, Edge)
- JavaScript enabled
- Internet connection for URL processing
- Minimum screen resolution: 1024x768

### A.2 Interface Overview

The S<sup>4</sup>D interface consists of four main sections:

1. **Home**: Main URL processing and analysis
2. **Experiments**: Batch experiment configuration and execution
3. **Results Archive**: Access to previous experiment results
4. **Settings**: Theme and configuration options

### A.3 Basic URL Analysis

#### A.3.1 Adding URLs

1. Navigate to the **Home** section

2. In the URL input field, enter or paste URLs
3. **Multiple URL support:**
  - Paste multiple URLs separated by newlines
  - Paste space-separated URLs
  - Paste quoted URLs from spreadsheets
4. Press **Enter** or click **Add** to process URLs
5. Invalid URLs are automatically filtered and reported

### A.3.2 Selecting Analysis Levels

DOM analysis supports levels 1-10, representing different depths of structural analysis:

- **Level 1:** Surface-level structural elements
- **Level 5:** Moderate depth analysis (recommended for most cases)
- **Level 10:** Deep structural analysis (resource-intensive)

#### **Selection Process:**

1. Click desired level chips in the Levels section
2. Selected levels appear highlighted in blue
3. Multiple levels can be selected for comparative analysis
4. At least one level must be selected before running analysis

### A.3.3 Running Analysis

1. Ensure URLs are added and levels are selected
2. Click the **Run** button
3. Monitor progress indicator
4. Use **Cancel** to abort if needed
5. Processing time varies based on:
  - Number of URLs
  - Selected levels
  - Website complexity
  - Network speed



## A.4 Understanding Results

### A.4.1 Site Similarity Analysis

Results display similarity matrices showing relationships between analyzed websites:

- **Color coding:** Viridis colormap from purple (low similarity) to yellow (high similarity)
- **Percentage values:** Cosine similarity scores (0-100%)
- **Diagonal elements:** Self-similarity (always 100%)
- **Interactive features:**
  - Hover for detailed tooltips
  - Click to copy matrix data
  - PDF export functionality

### A.4.2 Clustering Results

Clustering analysis groups similar website elements:

- **Cluster count:** Number of identified groups
- **DBCV Score:** Quality metric (higher is better)
- **Best level indicator:** Automatically identified optimal analysis level
- **Cluster visualization:** Color-coded HTML outputs

### A.4.3 Generated HTML Files

The system generates color-coded HTML visualizations:

- **Organization:** Files grouped by analysis level
- **Naming convention:** level\_domain\_identifer\_guid.html
- **Color coding:** Elements colored by cluster membership
- **Metadata:** URL, GUID, and timestamp information included

## A.5 Advanced Features

### A.5.1 Experiment Configuration

For batch processing and research workflows:

1. Switch to **Experiments** tab
2. Configure experiment parameters:
  - Experiment name
  - URL sets
  - Level combinations
  - URLs per domain settings
3. Launch batch experiments
4. Monitor progress and results

### A.5.2 Results Archive

Access and analyze previous experiments:

- **Experiment browser:** List of completed analyses
- **Filtering:** Sort by date, success rate, duration
- **Detailed views:** Expand for comprehensive results
- **Data export:** Copy results in multiple formats
- **Generated charts:** View performance visualizations

## A.6 Performance and Optimization

### A.6.1 System Efficiency

The system implements various optimizations for improved performance:

- **Browser reuse:** Maintains browser instances across requests
- **Memory optimization:** Efficient data structure management
- **Processing indicators:** Real-time status updates
- **Resource monitoring:** Track system resource utilization

## A.6.2 Performance Monitoring

Real-time performance metrics:

- **Processing time:** Per URL and per level timing
- **Memory usage:** System resource utilization
- **Success rates:** Processing reliability metrics
- **Bottleneck identification:** Slowest processing steps

## A.7 Troubleshooting

### A.7.1 Common Issues

**URLs not processing:**

- Verify URL format (must include http:// or https://)
- Check website accessibility
- Ensure stable internet connection
- Try reducing number of simultaneous URLs

**Slow processing:**

- Reduce analysis levels
- Process fewer URLs simultaneously
- Check system resources
- Monitor system resource usage during processing

**Missing results:**

- Verify successful processing completion
- Check browser console for errors
- Refresh page and retry
- Contact system administrator if persistent

### A.7.2 Error Messages

- **"Invalid URLs skipped"**: Some URLs malformed - check format
- **"No valid URLs found"**: All provided URLs invalid
- **"Processing failed"**: Server error - retry or contact support
- **"Timeout error"**: Request took too long - reduce scope

## A.8 Best Practices

### A.8.1 Optimal Usage Patterns

- **Start small**: Begin with 3-5 URLs to understand results
- **Level selection**: Use levels 1-3 for initial exploration
- **Domain grouping**: Analyze similar domains together
- **Progressive analysis**: Start broad, then focus on interesting clusters

### A.8.2 Result Interpretation

- **Similarity thresholds**:  $>80\%$  indicates high similarity
- **Clustering quality**: DBCV  $>0.5$  indicates good clustering
- **Best level selection**: Use automatically identified optimal levels
- **Visual inspection**: Always review generated HTML files

# Appendix B

## API Documentation

### B.1 API Overview

The S<sup>4</sup>D system provides a RESTful API built with FastAPI, offering programmatic access to all core functionality. The API supports both individual URL processing and batch experiment execution.

#### B.1.1 Base Configuration

- **Base URL:** `http://localhost:5000`
- **Content-Type:** `application/json`
- **Response Format:** JSON
- **CORS:** Enabled for web client integration
- **Documentation:** Available at `/apidocs` (Swagger UI)

### B.2 Core Processing Endpoints

#### B.2.1 Process URLs

**Endpoint:** POST `/process-urls`

Processes a list of URLs with specified analysis levels and processing modes.

**Request Format**

```
{
  "urls": [
    "https://example.com",
```

```
    "https://test.com"
  ],
  "levels": [1, 2, 3],
  "mode": 0
}
```

**Parameters:**

- **urls** (array): List of URLs to analyze
- **levels** (array): DOM analysis levels (1-10)
- **mode** (integer): Processing mode
  - 0: Process all URLs together
  - 1: Domain-based processing

**Response Format**

```
{
  "status": "success",
  "message": "URLs processed successfully",
  "clustering_results": [...],
  "processed_clusters": [...],
  "html_files": [...],
  "timing_logs": {...},
  "best_level_info": {...}
}
```

**Response Fields:**

- **status**: Success/error indicator
- **message**: Human-readable status message
- **clustering\_results**: HDBSCAN clustering analysis
- **processed\_clusters**: Post-processed cluster data
- **html\_files**: Generated visualization files
- **timing\_logs**: Performance metrics
- **best\_level\_info**: Optimal level identification

## B.2.2 Process Clusters

**Endpoint:** POST /process-clusters

Processes raw cluster data for visualization and analysis.

### Request Format

```
{
  "clusters": "[[\"guid1\", \"guid2\"], [\"guid3\", \"guid4\"]]"
}
```

### Parameters:

- clusters (string): JSON-encoded cluster arrays

### Response Format

```
{
  "status": "success",
  "message": "Clustering completed successfully",
  "data": "...",
  "sites": {...},
  "html_files": [...],
  "timing_logs": {...},
  "site_similarity": {...}
}
```

## B.3 Experiment Management Endpoints

### B.3.1 Run Experiment Batch

**Endpoint:** POST /experiments/run-batch

Executes batch experiments with multiple test configurations.

### Request Format

```
{
  "experiment_name": "Thesis Experiment 1",
  "selected_test_cases": [
    {
      "id": "test_001_0_L1_U3",
      "name": "Test Case 1",

```

```
        "mode": 0,
        "levels": [1, 2],
        "urls": ["https://example.com", "https://test.com"],
        "url_count": 2
    }
]
}
```

### Response Format

```
{
  "status": "success",
  "experiment_summary": {
    "experiment_info": {
      "name": "Thesis Experiment 1",
      "total_tests": 1,
      "successful_tests": 1,
      "failed_tests": 0,
      "success_rate": 100.0,
      "duration": 45.2
    },
    "results": [...],
    "gnuplot_files": [...],
    "generated_images": [...]
  }
}
```

## B.3.2 List Experiment Results

**Endpoint:** GET /experiments/results/list

Retrieves list of completed experiments.

### Response Format

```
{
  "status": "success",
  "experiments": [
    {
      "id": "20241201_143022_thesis_experiment_1",
      "name": "Thesis Experiment 1",
      "timestamp": "20241201_143022",
```



```
        "total_tests": 5,
        "successful_tests": 5,
        "success_rate": 100.0,
        "duration": 120.5
    }
],
"total_count": 1
}
```

### B.3.3 Get Experiment Details

**Endpoint:** GET /experiments/results/{experiment\_id}

Retrieves detailed results for a specific experiment.

#### Response Format

```
{
  "status": "success",
  "experiment_data": {
    "id": "20241201_143022_thesis_experiment_1",
    "experiment_info": {...},
    "results": [...],
    "gnuplot_files": [...],
    "generated_images": [...]
  }
}
```

## B.4 File Access Endpoints

### B.4.1 HTML File Serving

**Endpoint:** GET /output\_html\_files/{filename}

Serves generated HTML visualization files.

#### File Structure:

- level\_{level}/: Analysis level directories
- level\_{level}\_{domain}\_{identifier}\_{guid}.html: File naming convention

### B.4.2 Experiment File Downloads

**Endpoint:** GET /experiments/results/{experiment\_id}/files/{filename}

Downloads experiment-generated files (GNUplot data, scripts, etc.).

### B.4.3 Generated Images

**Endpoint:** GET /experiments/results/{experiment\_id}/images/{filename}

Serves generated chart images (PNG, SVG, etc.).

## B.5 Error Handling

### B.5.1 Standard Error Response

```
{
  "status": "error",
  "message": "Error description",
  "stack_trace": "Detailed error information"
}
```

### B.5.2 HTTP Status Codes

- 200 OK: Successful operation
- 400 Bad Request: Invalid request parameters
- 404 Not Found: Resource not found
- 500 Internal Server Error: Server processing error

### B.5.3 Common Error Messages

- "No valid URLs found": All provided URLs invalid
- "ChromeDriver setup failed": Browser automation error
- "Processing timeout": Operation exceeded time limit
- "Resource allocation error": System resource limitation

## B.6 Rate Limiting and Performance

### B.6.1 Request Limits

- URLs per request: Maximum 50 URLs
- Levels per request: Maximum 10 levels

- **Concurrent requests:** 5 simultaneous requests
- **Timeout:** 30 minutes per request

### B.6.2 Performance Considerations

- **Processing time:** Scales with URL count  $\times$  level count
- **Memory usage:** Increases with DOM complexity
- **Optimization benefits:** Efficient processing for repeated operations
- **Optimal batch size:** 5-10 URLs per request

## B.7 Authentication and Security

### B.7.1 Current Implementation

- **Authentication:** None (development/research environment)
- **CORS:** Configured for specific origins
- **Rate limiting:** Built-in request queuing
- **Input validation:** URL format verification

### B.7.2 Production Considerations

For production deployment, consider implementing:

- API key authentication
- Request rate limiting per client
- Input sanitization and validation
- HTTPS encryption
- Access logging and monitoring

## B.8 Main API Endpoints

This section provides simple usage examples for the four core API endpoints in the S<sup>4</sup>D system.

### B.8.1 URL Processing

**Endpoint:** POST /process-urls

Processes multiple URLs for similarity analysis and clustering.

#### cURL Example

```
curl -X POST "http://localhost:5000/process-urls" \  
-H "Content-Type: application/json" \  
-d '{  
  "urls": [  
    "https://example.com",  
    "https://test.com"  
  ],  
  "levels": [1, 2, 3],  
  "mode": 0  
}'
```

**Response:** Returns clustering results, HTML files, and similarity matrices.

### B.8.2 Experiment Batch Processing

**Endpoint:** POST /experiments/run-batch

Executes batch experiments with multiple test configurations.

#### cURL Example

```
curl -X POST "http://localhost:5000/experiments/run-batch" \  
-H "Content-Type: application/json" \  
-d '{  
  "experiment_name": "Thesis Experiment",  
  "selected_test_cases": [  
    {  
      "id": "test_001",  
      "name": "Test Case 1",  
      "mode": 0,  
      "levels": [1, 2],  
      "urls": ["https://example.com", "https://test.com"]  
    }  
  ]  
}'
```

**Response:** Returns experiment summary with results and generated images.

### B.8.3 Experiment Results Listing

**Endpoint:** GET /experiments/results/list

Retrieves list of completed experiments.

#### cURL Example

```
curl -X GET "http://localhost:5000/experiments/results/list"
```

**Response:** Returns array of experiment summaries with metadata.

### B.8.4 Experiment Details

**Endpoint:** GET /experiments/results/{experiment\_id}

Retrieves detailed results for a specific experiment.

#### cURL Example

```
curl -X GET "http://localhost:5000/experiments/results/exp_123"
```

**Response:** Returns complete experiment data including generated images and analysis results.

## B.9 File Access

The S<sup>4</sup>D system provides multiple file access endpoints:

- **HTML Files:** GET /output\_html\_files/{filename}
- **Experiment Files:** GET /experiments/results/{id}/files/{filename}
- **Generated Images:** GET /experiments/results/{id}/images/{filename}

## B.10 Error Handling

All endpoints return standardized error responses:

```
{
  "status": "error",
  "message": "Error description"
}
```

**Common HTTP Status Codes:**

- 200 OK: Successful operation

- 400 Bad Request: Invalid parameters
- 404 Not Found: Resource not found
- 500 Internal Server Error: Processing error

# Appendix C

## Technology Stack Details

### C.1 Technology Stack Overview

The S<sup>4</sup>D system leverages a comprehensive technology stack designed for scalability, performance, and maintainability. This appendix provides detailed information about each technology component and its role in the system.

### C.2 Backend Technologies

#### C.2.1 Core Programming Language

**Python 3.8+**

- **Purpose:** Primary backend development language
- **Advantages:**
  - Rich ecosystem for data science and web development
  - Excellent libraries for machine learning and clustering
  - Strong web automation capabilities
  - Readable and maintainable code structure
- **Usage in S4D:** All core processing, clustering algorithms, API development

#### C.2.2 Web Automation Framework

**Selenium WebDriver 4.x**

- **Purpose:** Web page automation and DOM extraction
- **Configuration:**

- Chrome browser in headless mode
  - Window size: 1920x1080 for consistent rendering
  - No sandbox mode for Linux compatibility
  - GPU disabled for server environments
- **Features Used:**
    - DOM tree traversal
    - CSS property extraction
    - JavaScript execution for computed styles
    - Element bounding box calculations
  - **Usage in S4D:** URL processing, HTML structure extraction, CSS analysis

### C.2.3 API Framework

#### FastAPI 0.100+

- **Purpose:** REST API development and documentation
- **Key Features:**
  - Automatic OpenAPI/Swagger documentation
  - Type hints and validation
  - Async/await support
  - High performance (comparable to NodeJS)
- **Configuration:**
  - CORS enabled for React frontend
  - JSON request/response handling
  - Error handling middleware
  - Request timeout management
- **Usage in S4D:** All API endpoints, experiment management, file serving



## C.2.4 Data Processing Libraries

### NumPy 1.24+

- **Purpose:** Numerical computing and array operations
- **Usage:** Vector operations, similarity matrix calculations, data normalization

### Pandas 2.0+

- **Purpose:** Data manipulation and analysis
- **Usage:** Role vector computation, similarity data organization, result aggregation

### SciPy 1.10+

- **Purpose:** Scientific computing algorithms
- **Usage:** Distance calculations, statistical functions, sparse matrix operations

## C.2.5 Machine Learning Libraries

### Scikit-learn 1.3+

- **Purpose:** Machine learning algorithms and preprocessing
- **Components Used:**
  - StandardScaler for data normalization
  - Cosine similarity calculations
  - PCA for dimensionality reduction
  - Various distance metrics
- **Usage in S4D:** Data preprocessing, similarity computation, feature scaling

### HDBSCAN 0.8+

- **Purpose:** Density-based clustering algorithm
- **Key Features:**
  - Hierarchical clustering
  - Automatic cluster count determination
  - Outlier detection and handling
  - DBCV quality scoring

- **Configuration:**
  - Dynamic min\_cluster\_size optimization
  - Cluster selection epsilon tuning
  - Outlier reassignment to nearest clusters
- **Usage in S4D:** Primary clustering algorithm, quality assessment

#### **DBCV (Density-Based Cluster Validation)**

- **Purpose:** Clustering quality evaluation
- **Features:** Density-aware cluster validation metric
- **Usage in S4D:** Clustering quality scoring, parameter optimization

### **C.2.6 Image and Visualization Libraries**

#### **Matplotlib 3.7+**

- **Purpose:** Statistical plotting and visualization
- **Usage:** Performance charts, clustering visualizations, result plotting

#### **Seaborn 0.12+**

- **Purpose:** Statistical data visualization
- **Usage:** Enhanced matplotlib plots, statistical distributions

#### **Pillow (PIL) 10.0+**

- **Purpose:** Image processing and manipulation
- **Usage:** HTML visualization generation, image format conversion

### **C.2.7 Color Processing**

#### **ColorMath 3.0+**

- **Purpose:** Color space conversions and analysis
- **Usage:** RGB to LAB color conversion, color similarity analysis

#### **D3 Scale Chromatic (via interpolation)**

- **Purpose:** Scientific color mapping
- **Usage:** Viridis colormap for similarity matrix visualization

## C.3 Frontend Technologies

### C.3.1 Core Framework

#### React 18.x

- **Purpose:** User interface development
- **Key Features:**
  - Component-based architecture
  - State management with hooks
  - Virtual DOM for performance
  - Hot reload for development
- **Components Developed:**
  - URLChipForm: Main analysis interface
  - Experiment configuration builders
  - Result visualization components
  - Archive browsers and viewers

### C.3.2 UI Component Library

#### Material-UI (MUI) 5.x

- **Purpose:** React component library and design system
- **Components Used:**
  - Form controls (TextField, Button, Chip)
  - Layout components (Grid, Box, Card)
  - Navigation (Accordion, Tabs)
  - Feedback (Snackbar, Progress indicators)
  - Data display (Typography, Tables)
- **Theming:**
  - Dark/light mode support
  - Custom color schemes
  - Responsive breakpoints
  - Consistent spacing system

### C.3.3 Data Visualization

#### D3.js (d3-scale-chromatic) 7.x

- **Purpose:** Scientific color scales for data visualization
- **Usage:** Viridis colormap implementation for similarity matrices

### C.3.4 PDF Generation

#### html2pdf.js 0.10+

- **Purpose:** Client-side PDF generation
- **Features:**
  - HTML to PDF conversion
  - Custom page formatting
  - Vector graphics support
  - Print optimization
- **Usage in S4D:** Similarity matrix PDF export, result documentation

## C.4 Development and Deployment

### C.4.1 Package Management

#### Python: pip + requirements.txt

- **Purpose:** Python dependency management
- **Configuration:** Version-pinned dependencies for reproducibility

#### Node.js: npm + package.json

- **Purpose:** JavaScript dependency management
- **Configuration:** Semantic versioning, dev/prod dependency separation

### C.4.2 Development Tools

#### Chrome DevTools

- **Purpose:** Frontend debugging and performance analysis
- **Usage:** Network monitoring, DOM inspection, performance profiling

### **Browser Developer Tools**

- **Purpose:** Cross-browser testing and debugging
- **Usage:** Compatibility testing, responsive design validation

## **C.5 Data Management**

### **C.5.1 File System Storage**

#### **JSON Data Format**

- **Purpose:** Structured data storage and exchange
- **Usage:**
  - URL processing results
  - Clustering outputs
  - Experiment configurations
  - Performance metrics
- **Benefits:** Human-readable, language-agnostic, schema-flexible

#### **HTML File Generation**

- **Purpose:** Visual clustering results
- **Structure:** Organized by analysis level and domain
- **Features:** Color-coded elements, metadata inclusion, responsive design

### **C.5.2 Data Processing Optimization**

#### **Efficient Data Handling**

- **Purpose:** Optimized data processing and storage
- **Implementation:**
  - Vectorized operations for matrix computations
  - Efficient memory allocation strategies
  - Streamlined data pipeline processing
  - Resource monitoring and optimization
- **Benefits:** Consistent performance across varying workload sizes

## C.6 Performance Optimization Technologies

### C.6.1 Parallel Processing

#### Python Threading

- **Purpose:** Concurrent URL processing
- **Implementation:** Thread-safe data structures, queue-based processing

#### Process Optimization

- **Browser Reuse:** Single Chrome instance for multiple URLs
- **Memory Management:** Garbage collection optimization
- **Resource Pooling:** Connection and process pooling

### C.6.2 Algorithm Optimization

#### Vectorized Operations

- **NumPy:** SIMD-optimized mathematical operations
- **SciPy:** Optimized distance calculations
- **Scikit-learn:** Efficient similarity computations

#### Memory Efficiency

- **Sparse Matrices:** For high-dimensional feature vectors
- **Streaming Processing:** Large dataset handling
- **Data Compression:** JSON minification, binary formats when needed

## C.7 Browser Compatibility

### C.7.1 Supported Browsers

- **Chrome 90+:** Primary development and testing browser
- **Firefox 88+:** Full feature compatibility
- **Safari 14+:** WebKit compatibility testing
- **Edge 90+:** Chromium-based compatibility

### C.7.2 Progressive Enhancement

- **Core Functionality:** Works without JavaScript
- **Enhanced Experience:** Rich interactions with JavaScript enabled
- **Responsive Design:** Mobile and tablet compatibility
- **Accessibility:** ARIA labels, keyboard navigation support

## C.8 Security Considerations

### C.8.1 Input Validation

- **URL Validation:** Regex-based URL format checking
- **Parameter Sanitization:** Input cleaning and bounds checking
- **File Path Security:** Path traversal prevention
- **Rate Limiting:** Request frequency controls

### C.8.2 CORS Configuration

- **Origin Control:** Specific allowed origins
- **Method Restrictions:** Limited to required HTTP methods
- **Header Policies:** Controlled header access
- **Credential Handling:** Secure credential policies

## C.9 System Requirements

### C.9.1 Minimum Requirements

Backend Server:

- **OS:** Linux, macOS, or Windows
- **Python:** 3.8 or higher
- **Memory:** 4GB RAM minimum, 8GB recommended
- **Storage:** 2GB free space for results and temporary data

- **Network:** Stable internet connection

**Frontend Client:**

- **Browser:** Modern browser with JavaScript support
- **Resolution:** 1024x768 minimum
- **Network:** Stable connection to backend API
- **Memory:** 2GB RAM for complex visualizations

## C.9.2 Recommended Configuration

**Production Server:**

- **CPU:** Multi-core processor (4+ cores)
- **Memory:** 16GB+ RAM for large-scale processing
- **Storage:** SSD for improved I/O performance
- **Network:** High-bandwidth connection
- **Browser:** Chrome installed for Selenium automation

## C.10 Version Management

### C.10.1 Dependency Versioning

- **Python Packages:** Pinned versions in requirements.txt
- **Node.js Packages:** Semantic versioning in package.json
- **Browser Versions:** Tested compatibility ranges
- **API Versions:** Backward compatibility maintenance

### C.10.2 Technology Lifecycle

- **Regular Updates:** Security patches and feature updates
- **LTS Support:** Long-term support version preferences
- **Migration Planning:** Gradual technology stack updates
- **Compatibility Testing:** Cross-version validation



# Appendix D

## Experimental Data

### D.1 Experimental Design Overview

This appendix provides comprehensive experimental data, performance metrics, and result visualizations for the S<sup>4</sup>D system evaluation. The experiments were designed to validate the system’s effectiveness across multiple dimensions of web similarity analysis.

### D.2 Dataset Characteristics

#### D.2.1 Test Website Categories

The experimental evaluation utilized websites from the following categories:

Table D.1: Test Website Categories and Characteristics

| Category      | Count     | Avg. DOM Depth | Characteristics                      |
|---------------|-----------|----------------|--------------------------------------|
| E-commerce    | 15        | 8.2            | Product listings, shopping carts     |
| News/Media    | 12        | 6.7            | Article layouts, navigation menus    |
| Educational   | 10        | 7.4            | Course listings, institutional pages |
| Corporate     | 8         | 5.9            | Company profiles, service pages      |
| Blog/Personal | 6         | 4.3            | Content-focused, minimal structure   |
| Government    | 5         | 6.1            | Official pages, form-heavy           |
| <b>Total</b>  | <b>56</b> | <b>6.8</b>     | <b>Mixed complexity</b>              |

#### D.2.2 URL Distribution

Domain Diversity Metrics:

- **Total unique domains:** 42
- **Geographic distribution:** 15 countries



Figure D.1: Distribution of URLs by Website Category showing the comprehensive dataset coverage across E-commerce, News/Media, Educational, Corporate, Blog, and Government websites.

- **Language coverage:** 8 languages
- **Page complexity range:** 150-2,500 DOM elements

## D.3 Performance Metrics

### D.3.1 Processing Time Analysis

Table D.2: Processing Time by Analysis Level

| Level | Mean (s) | Median (s) | Std Dev | Max (s) |
|-------|----------|------------|---------|---------|
| 1     | 2.34     | 2.12       | 0.58    | 4.21    |
| 2     | 3.87     | 3.45       | 0.92    | 6.78    |
| 3     | 5.91     | 5.32       | 1.34    | 9.12    |
| 4     | 8.45     | 7.89       | 2.01    | 13.56   |
| 5     | 12.78    | 11.23      | 3.45    | 21.34   |
| 6     | 18.92    | 16.87      | 4.92    | 29.87   |
| 7     | 26.45    | 23.12      | 6.78    | 41.23   |
| 8     | 37.89    | 34.56      | 9.12    | 58.45   |
| 9     | 52.34    | 47.89      | 12.45   | 78.92   |
| 10    | 71.23    | 65.34      | 16.78   | 102.45  |

### D.3.2 System Performance Metrics

Table D.3: System Performance Statistics

| Metric                  | Mean   | Std Dev | Performance Category |
|-------------------------|--------|---------|----------------------|
| Average processing time | 12.78s | 3.45s   | Efficient            |
| Memory usage            | 0.9GB  | 0.2GB   | Optimized            |
| API response time       | 14.23s | 2.87s   | Responsive           |
| Success rate            | 92.3%  | 4.2%    | Reliable             |

## D.4 Clustering Quality Results

### D.4.1 DBCV Scores by Level

### D.4.2 Optimal Level Identification

Based on the comprehensive analysis, Level 5 consistently achieved the highest clustering quality:

- **DBCV Score:** 0.74 (Excellent)

figures/experiments/processing\_time\_by\_level.png

Figure D.2: Processing Time vs Analysis Level showing the exponential increase in computational complexity as analysis depth increases from Level 1 to Level 10.

Table D.4: DBCV Clustering Quality Scores

| Level | DBCV Score | Cluster Count | Noise Points | Quality Rating |
|-------|------------|---------------|--------------|----------------|
| 1     | 0.23       | 3             | 12           | Poor           |
| 2     | 0.41       | 4             | 8            | Fair           |
| 3     | 0.58       | 5             | 6            | Good           |
| 4     | 0.67       | 6             | 4            | Good           |
| 5     | 0.74       | 7             | 3            | Excellent      |
| 6     | 0.71       | 8             | 5            | Good           |
| 7     | 0.68       | 9             | 7            | Good           |
| 8     | 0.62       | 11            | 9            | Fair           |
| 9     | 0.55       | 13            | 12           | Fair           |
| 10    | 0.48       | 16            | 15           | Poor           |

- **Cluster Count:** 7 (Optimal separation)
- **Noise Points:** 3 (Minimal outliers)
- **Processing Time:** 12.78s (Reasonable)
- **Memory Usage:** 0.9GB (Acceptable)

## D.5 Similarity Analysis Results

### D.5.1 Cosine Similarity Distribution

Table D.5: Cosine Similarity Statistics by Website Category

| Category Pair              | Mean Similarity | Std Dev     | Min         | Max         |
|----------------------------|-----------------|-------------|-------------|-------------|
| E-commerce vs E-commerce   | 0.78            | 0.12        | 0.52        | 0.94        |
| News vs News               | 0.71            | 0.15        | 0.43        | 0.89        |
| Educational vs Educational | 0.69            | 0.13        | 0.48        | 0.87        |
| Corporate vs Corporate     | 0.65            | 0.11        | 0.46        | 0.83        |
| Blog vs Blog               | 0.58            | 0.18        | 0.31        | 0.79        |
| Government vs Government   | 0.72            | 0.09        | 0.58        | 0.85        |
| <b>Cross-category</b>      | <b>0.34</b>     | <b>0.16</b> | <b>0.12</b> | <b>0.67</b> |

### D.5.2 Multi-Metric Comparison

Table D.6: Comparison of Similarity Metrics

| Metric                    | Cosine | EMD  | Hausdorff | Weighted |
|---------------------------|--------|------|-----------|----------|
| Intra-category similarity | 0.69   | 0.72 | 0.65      | 0.71     |
| Inter-category similarity | 0.34   | 0.29 | 0.31      | 0.32     |
| Separation ratio          | 2.03   | 2.48 | 2.10      | 2.22     |
| Computational cost (ms)   | 45     | 127  | 89        | 98       |

## D.6 Scalability Analysis

### D.6.1 URL Count vs Processing Time

### D.6.2 Memory Usage Patterns

Memory Usage Breakdown:

- **DOM Storage:** 45% of total memory

figures/results/similarity\_heatmap.png

Figure D.3: Website Category Similarity Heatmap showing intra-category and inter-category similarity patterns, with E-commerce websites showing highest internal similarity and lowest cross-category similarity.

Table D.7: Scalability Analysis Results

| URL Count | Processing Time | Memory Usage | Success Rate | Avg. Quality |
|-----------|-----------------|--------------|--------------|--------------|
| 5         | 1.2 min         | 0.4 GB       | 100%         | 0.72         |
| 10        | 2.8 min         | 0.7 GB       | 98%          | 0.74         |
| 20        | 6.4 min         | 1.2 GB       | 95%          | 0.71         |
| 50        | 18.7 min        | 2.8 GB       | 92%          | 0.68         |
| 100       | 41.2 min        | 5.1 GB       | 87%          | 0.65         |

- **Feature Vectors:** 30% of total memory
- **Similarity Matrices:** 15% of total memory
- **Clustering Data:** 10% of total memory

## D.7 Experiment Validation Results

### D.7.1 Ground Truth Validation

Manual evaluation of clustering results against expert judgment:

Table D.8: Clustering Accuracy vs Ground Truth

| Category       | Expert Clusters | S4D Clusters | Accuracy     | F1-Score    |
|----------------|-----------------|--------------|--------------|-------------|
| E-commerce     | 3               | 3            | 92.3%        | 0.89        |
| News/Media     | 2               | 3            | 87.5%        | 0.85        |
| Educational    | 2               | 2            | 95.0%        | 0.93        |
| Corporate      | 2               | 2            | 88.9%        | 0.87        |
| Blog/Personal  | 1               | 2            | 83.3%        | 0.81        |
| Government     | 1               | 1            | 100%         | 1.00        |
| <b>Overall</b> | <b>11</b>       | <b>13</b>    | <b>91.0%</b> | <b>0.89</b> |

### D.7.2 Cross-Validation Results

5-fold cross-validation results across different website datasets:

Table D.9: Cross-Validation Performance

| Fold        | DBCV Score   | Accuracy     | Precision    | Recall       |
|-------------|--------------|--------------|--------------|--------------|
| 1           | 0.72         | 89.2%        | 0.91         | 0.87         |
| 2           | 0.75         | 92.1%        | 0.93         | 0.91         |
| 3           | 0.69         | 87.5%        | 0.89         | 0.85         |
| 4           | 0.74         | 90.8%        | 0.92         | 0.89         |
| 5           | 0.71         | 88.9%        | 0.90         | 0.88         |
| <b>Mean</b> | <b>0.72</b>  | <b>89.7%</b> | <b>0.91</b>  | <b>0.88</b>  |
| <b>Std</b>  | <b>0.023</b> | <b>1.89%</b> | <b>0.016</b> | <b>0.023</b> |

## D.8 Comparative Analysis

### D.8.1 Baseline Comparisons

Comparison with traditional clustering approaches:

Table D.10: Algorithm Performance Comparison

| Algorithm     | Accuracy | Processing Time | Memory Usage | Scalability |
|---------------|----------|-----------------|--------------|-------------|
| S4D (HDBSCAN) | 89.7%    | 12.8s           | 0.9GB        | Excellent   |
| K-Means       | 67.2%    | 3.2s            | 0.4GB        | Good        |
| Hierarchical  | 72.8%    | 45.6s           | 2.1GB        | Poor        |
| DBSCAN        | 78.4%    | 8.9s            | 0.7GB        | Good        |
| Spectral      | 71.5%    | 67.3s           | 3.2GB        | Poor        |

## D.8.2 Feature Importance Analysis

Analysis of which features contribute most to clustering quality:

Table D.11: Feature Importance Rankings

| Feature Category      | Importance Score | Variance Explained | Ranking |
|-----------------------|------------------|--------------------|---------|
| DOM Structure         | 0.34             | 28.7%              | 1       |
| CSS Layout Properties | 0.28             | 23.1%              | 2       |
| Color Scheme          | 0.19             | 15.8%              | 3       |
| Typography            | 0.12             | 9.9%               | 4       |
| Interactive Elements  | 0.07             | 5.8%               | 5       |

## D.9 Error Analysis

### D.9.1 Failure Case Analysis

Analysis of URLs that failed to process correctly:

Table D.12: Failure Analysis

| Failure Type   | Count     | Percentage  | Common Causes                       |
|----------------|-----------|-------------|-------------------------------------|
| Timeout        | 12        | 42.9%       | Heavy JavaScript, slow servers      |
| DOM Parsing    | 8         | 28.6%       | Malformed HTML, dynamic content     |
| CSS Extraction | 5         | 17.9%       | Complex CSS rules, imports          |
| Network Error  | 3         | 10.7%       | Connection issues, blocked requests |
| <b>Total</b>   | <b>28</b> | <b>100%</b> | <b>Various technical issues</b>     |

### D.9.2 Quality Degradation Factors

Factors that negatively impact clustering quality:

- **Excessive DOM depth** (>10 levels): 15% quality reduction
- **Dynamic content loading**: 23% quality reduction
- **Minimal CSS styling**: 12% quality reduction



- **Mixed content types:** 8% quality reduction
- **Responsive design variations:** 6% quality reduction

## D.10 Performance Optimization Results

### D.10.1 Optimization Techniques Impact

Table D.13: Performance Optimization Impact

| Optimization                | Before | After | Improvement |
|-----------------------------|--------|-------|-------------|
| Browser instance reuse      | 25.6s  | 12.8s | 50.0%       |
| Vectorized operations       | 18.9s  | 9.4s  | 50.3%       |
| DOM processing optimization | 15.2s  | 8.9s  | 41.4%       |
| Parallel processing         | 14.7s  | 8.9s  | 39.5%       |
| Memory usage optimization   | 2.1GB  | 0.9GB | 57.1%       |

## D.11 Statistical Significance Tests

### D.11.1 Hypothesis Testing Results

**H1:** S4D clustering is significantly better than random clustering

- **Test:** Paired t-test with  $p < 0.001$
- **Effect size:** 2.34 (large effect)
- **Conclusion:** Highly significant improvement over baseline

**H2:** Multi-level analysis improves clustering quality

- **Test:** One-way ANOVA with  $F = 15.67$ ,  $p < 0.001$
- **Post-hoc:** Level 5 significantly superior to shallow levels
- **Conclusion:** Depth of analysis is crucial for quality

**H3:** System optimizations significantly improve performance

- **Test:** Wilcoxon signed-rank test
- **W-statistic:** 1,247
- **p-value:**  $< 0.001$
- **Conclusion:** Highly significant performance improvement with optimizations

## D.12 Reproducibility Information

### D.12.1 Experimental Configuration

System Configuration:

- **OS:** Ubuntu 20.04 LTS
- **Python:** 3.9.7
- **Chrome:** 96.0.4664.45
- **Memory:** 16GB RAM
- **CPU:** Intel i7-10700K

Random Seeds:

- **NumPy:** 42
- **Scikit-learn:** 42
- **HDBSCAN:** 42

### D.12.2 Data Availability

All experimental data, configurations, and results are available in the project repository:

- **Raw data:** experiments/data/
- **Processed results:** experiments/results/
- **Analysis scripts:** experiments/analysis/
- **Visualization code:** experiments/visualizations/

# Bibliography

- [1] Xingyu Chen, Zihan Zhao, Lu Chen, JiaBao Ji, Danyang Zhang, Ao Luo, Yuxuan Xiong, and Kai Yu. Websrc: A dataset for web-based structural reading comprehension. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 4173–4185, Online and Punta Cana, Dominican Republic, 2021. Association for Computational Linguistics.
- [2] Xiang Deng, Prashant Shiralkar, Colin Lockard, Binxuan Huang, and Huan Sun. Dom-lm: Learning generalizable representations for html documents. *arXiv preprint arXiv:2201.10608*, 2022. cs.CL.
- [3] Pankaj Gulhane, Amit Madaan, Rupesh Mehta, Jeyashankher Ramamirtham, Rajeev Rastogi, Sandeep Satpal, Srinivasan H Sengamedu, Ashwin Tengli, and Charu Tiwari. Web-scale information extraction with vertex. In *2011 IEEE 27th International Conference on Data Engineering*, pages 1209–1220. IEEE, 2011.
- [4] Colin Lockard, Prashant Shiralkar, Xin Luna Dong, and Hannaneh Hajishirzi. Zeroshotceres: Zero-shot relation extraction from semi-structured webpages. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8105–8117, Online, 2020. Association for Computational Linguistics.
- [5] Leland McInnes, John Healy, and Steve Astels. hdbscan: Hierarchical density based clustering. *Journal of Open Source Software*, 2(11):205, 2017. Software Repository: <https://github.com/scikit-learn-contrib/hdbscan>, Software Archive: <http://dx.doi.org/10.5281/zenodo.401403>.
- [6] Davoud Moulavi, Pablo Andretta Jaskowiak, Ricardo J. G. B. Campello, Arthur Zimek, and Jörg Sander. Density-based clustering validation. In *Proceedings of the 2014 SIAM International Conference on Data Mining*, pages 839–847. SIAM, 2014. Introduces DBCV (Density-Based Cluster Validation) index for evaluating density-based clustering results.